Contents

Prerequisites

You will want to have Unity Hub downloaded and installed to follow along with this document. If you need help downloading the required software, check out the document *Installing Unity Hub, Editor Versions, & Visual Studio*.

Everything shown in this document was performed using Unity editor version 6000.3.9f1. If you want to follow along precisely, I recommend downloading that editor version (see Unity Hub Installs for setup). Additionally, the project shown uses the Universal 3D core rendering pipeline. To follow this documentation exactly, create a new Unity project using Universal 3D core as your rendering pipeline (see *Creating a New Project* below).

Introduction

Getting into Unity for the first time can be difficult. Unity's software is designed to break down many of the resources the engine offers. As a result, many useful settings and systems in Unity can feel hidden due to the sheer number of windows, unclear user actions, and Unity-specific terminology. This document will break down everything you need to know to navigate Unity, from Unity Hub to the editor.

Unity Hub

Unity Hub has undergone many changes over the years. However, the content within has remained relatively similar. As of writing this document, Unity Hub is broken up into five sections: Projects, Installs, Learn, Resources, and Licenses.

Projects

The project section of Unity Hub is the central location for all your Unity projects. In it, you can find the names of all your projects, their locations on your computer, the last time they were modified, and the editor version they were made in.

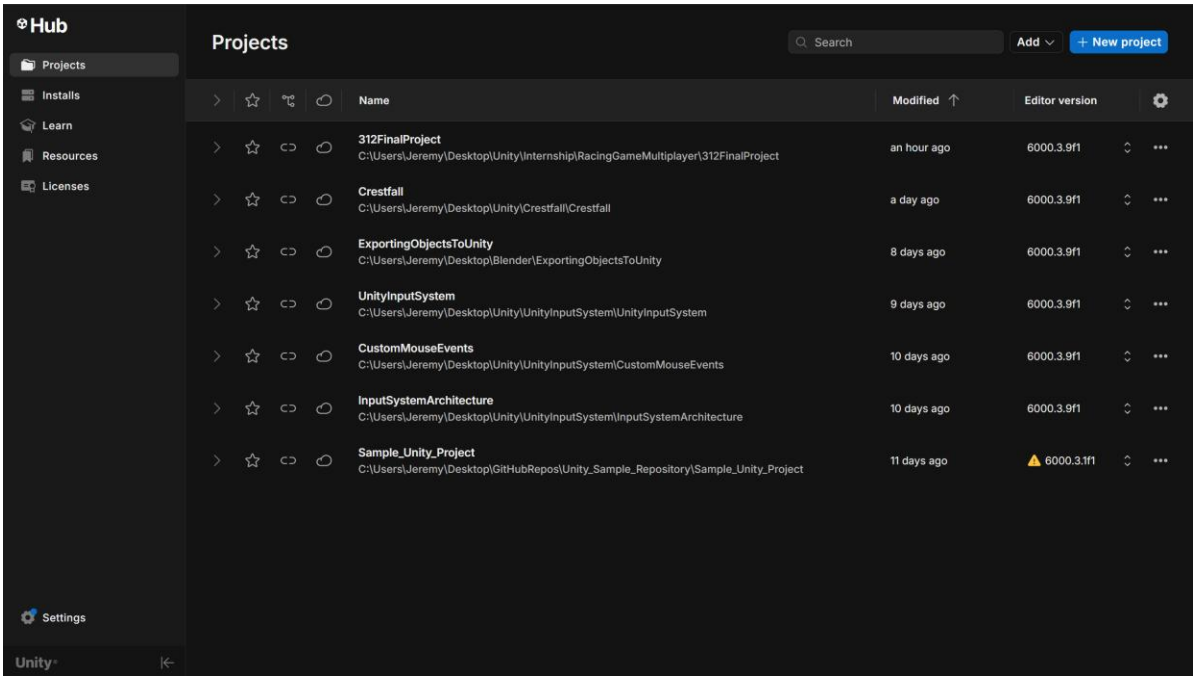

Figure 1 - Unity Hub Projects Tab View

You can view more properties for your project by clicking the arrow on the left-hand side of your project. Doing so will expand the project's properties to show the organization, cloud project, and source control repository.

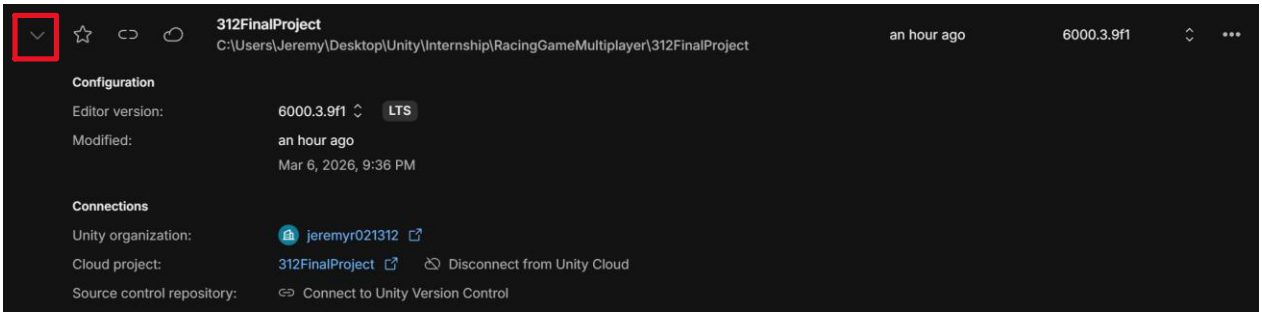

Figure 2 - Viewing Additional Information About Projects

Creating Unity Project

The project section lets you create a new Unity project. Clicking the New Project button in the top-right corner will open a new page where you can configure the following:

- The editor version the project will use
- The rendering pipeline template to start from
- The Unity organization your project will connect to
- Your project's name
- The location of the project on your computer

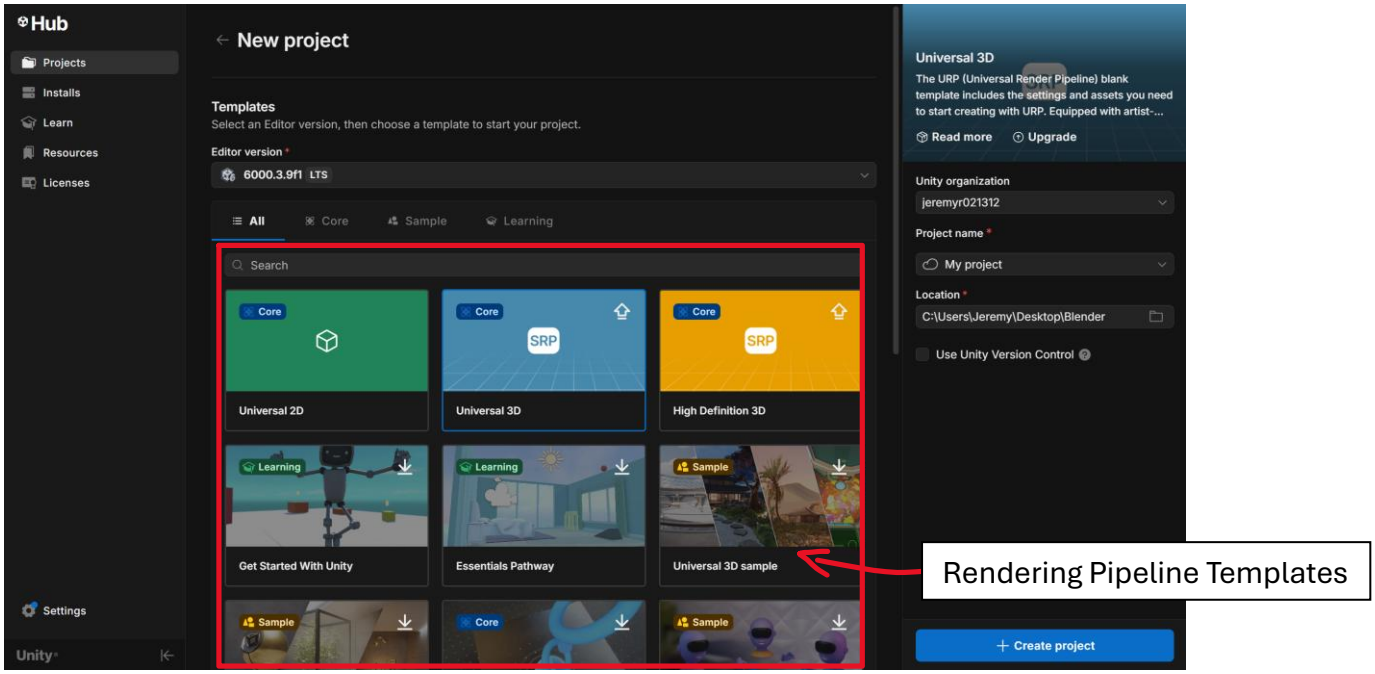

Figure 3 - Sample Setup for A New Unity Project

The main rendering pipelines to be aware of are the Universal 2D and 3D core pipelines. Use Universal 2D for 2D projects and Universal 3D for 3D projects. Unity will automatically set up your project environment accordingly.

Adding Project from Disk

If you have a project on your computer but it doesn't show in Unity Hub's project section, you can manually add the project from your disk:

1. Click Add -> Add Project from Disk.

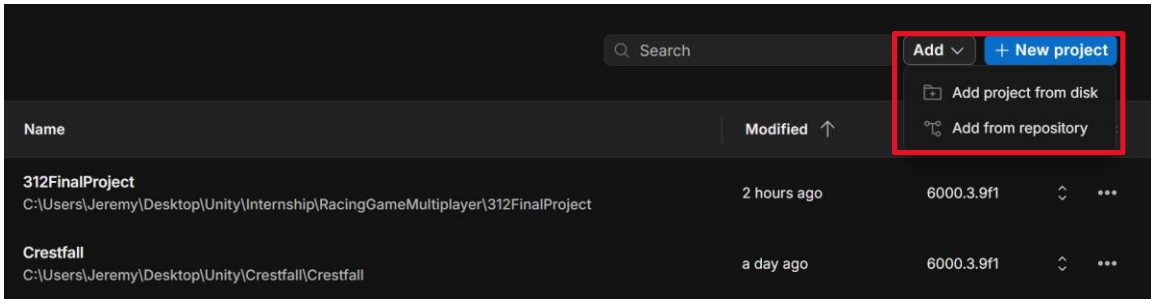

Figure 4 - Location for Where to Add Project From Disk

2. Find the location of your Unity *folder* in your files. You will know you reached the folder when the content within looks like the following:

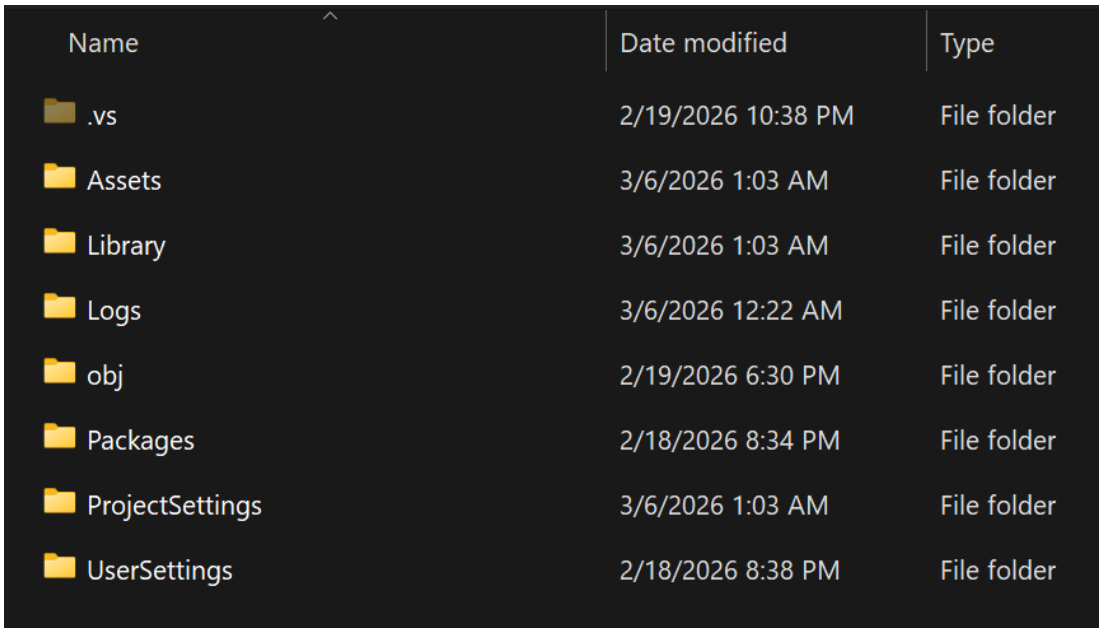

Figure 5 - Sample Unity Project Folder Hierarchy

   a. Note: Add the parent folder for the content above. That folder should be the name of your Unity project.

Installs

The installs section in Unity Hub shows all the editor versions installed on your computer. For each editor, you can see the name of the editor, the number of projects using it, and the modules installed on the editor (i.e., WebGL, Windows Build Support, etc.)

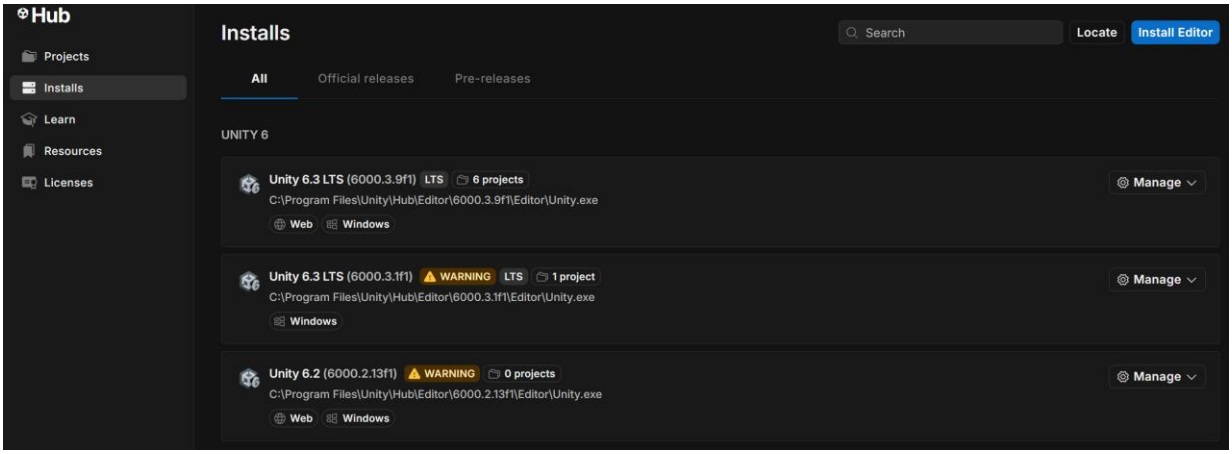

Figure 6 - Unity Hub Installs View

You can add modules to an existing editor version by clicking on the manage button on the right-hand side of the version. If there is a module you forgot to add when adding the editor version, this is how you can add the module.

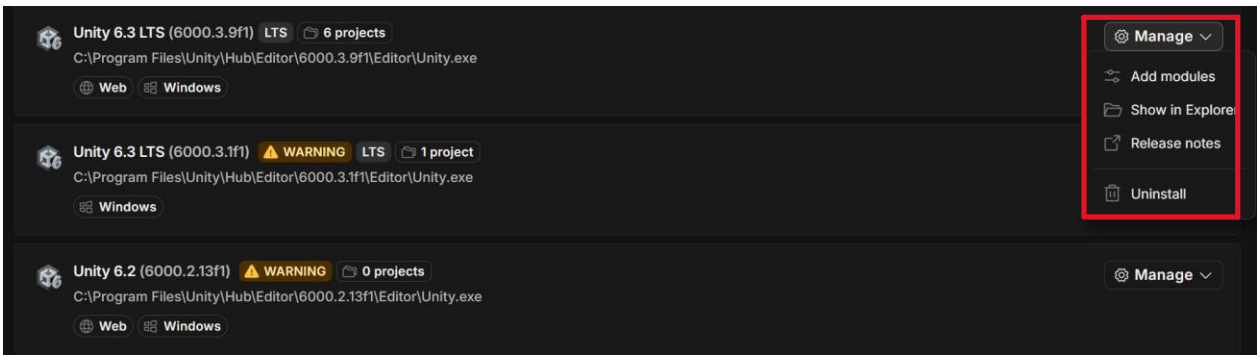

Figure 7 - Location of Editor Version Management Settings

Installing Official Editor Releases

Unity primarily shows you the official release versions when installing an editor. To install an editor, click the blue button at the top-right of the screen. You will see the following:

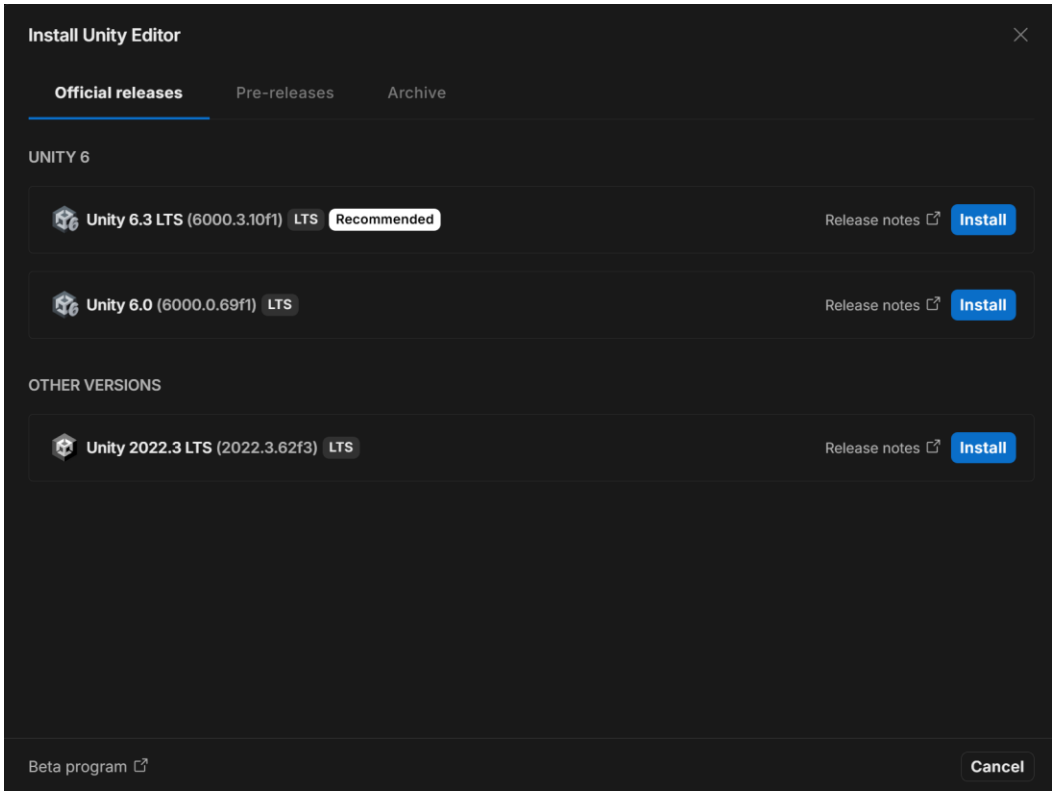

Figure 8 - Sample View of The Unity Editor Install Section

I recommend installing any version with long-term support (LTS). Upon selecting an editor version, Unity will ask you to select some modules, with web build support selected by default. After selecting your modules, click Install, and wait for Unity to download the editor.

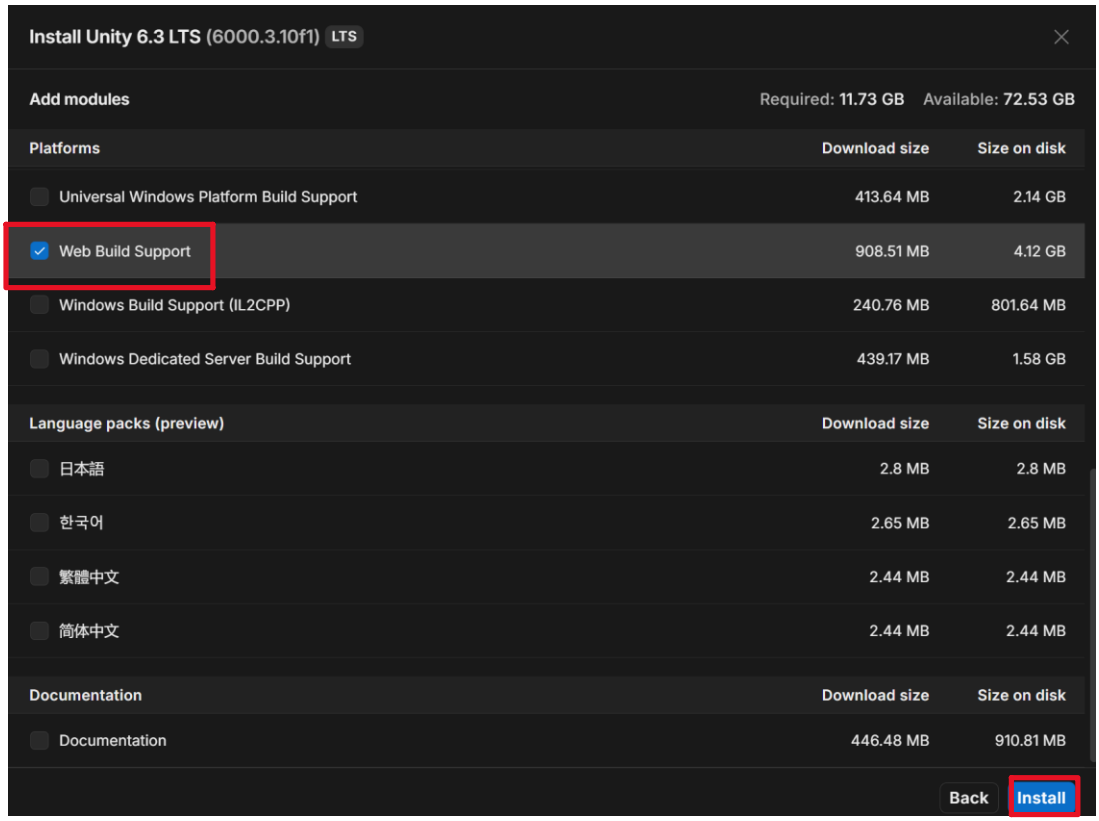

Figure 9 - Sample Modules To Install

Installing Archived Editors

If you want to install an old Unity editor version, you can find it on Unity's archive page. To do so:

1. Click Install Editor
2. Click on Archive. You will see a hyperlink to visit the download archive page. Click it.

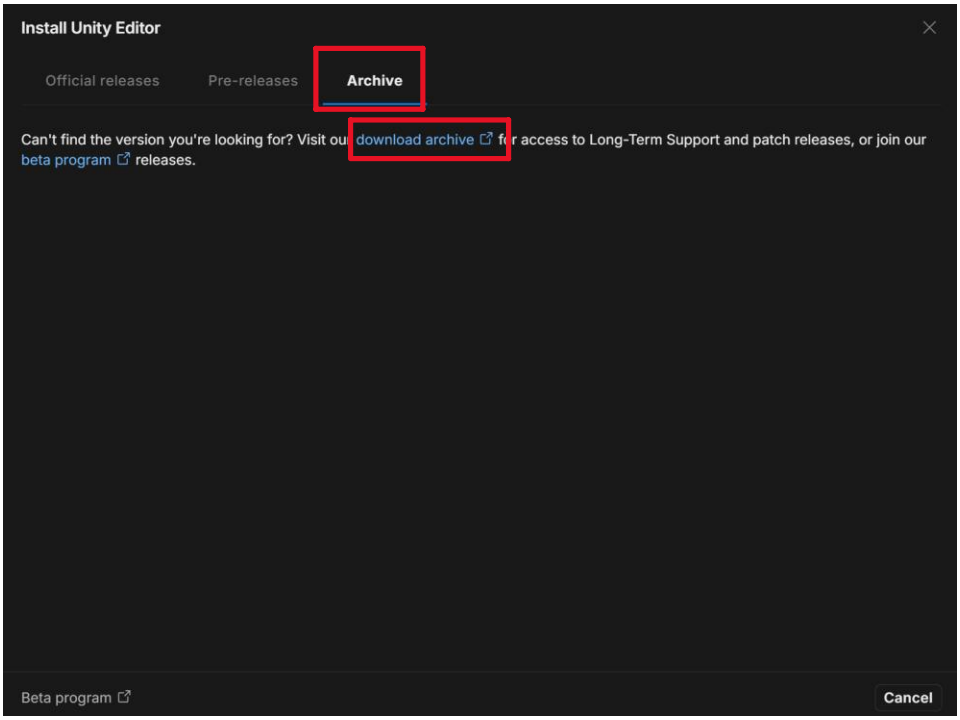

Figure 10 - Location for Where to Install Archived Editor Versions

3. Find the editor version you wish to install. Click the install button for the editor version, and, when prompted, open Unity Hub.
    a. Note: I highly recommend not installing an editor version with warnings on it, as the editor version may be unstable or put yourself at risk.

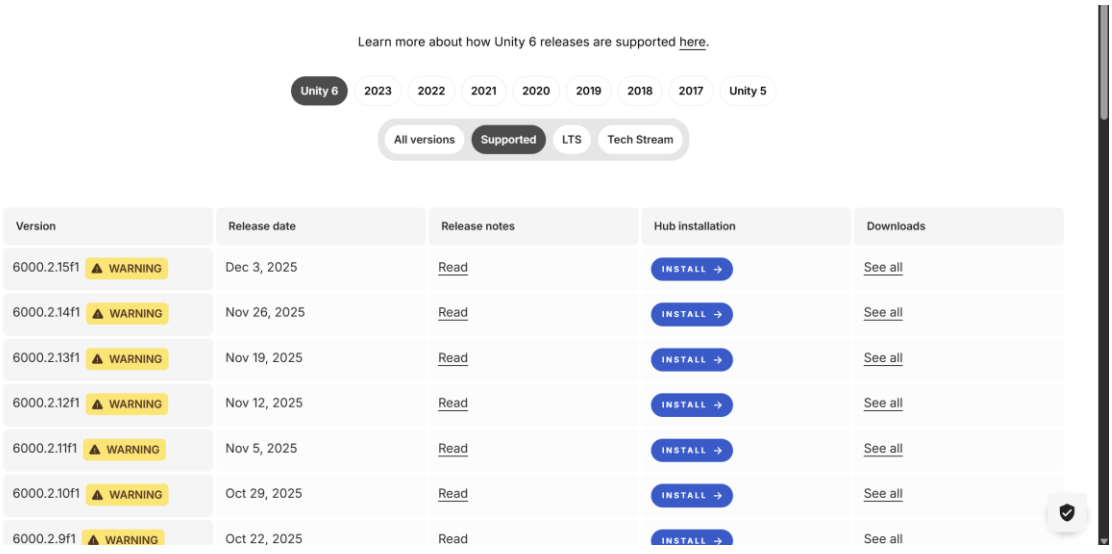

Figure 11 - Sample View of Unity's Editor Version Webpage

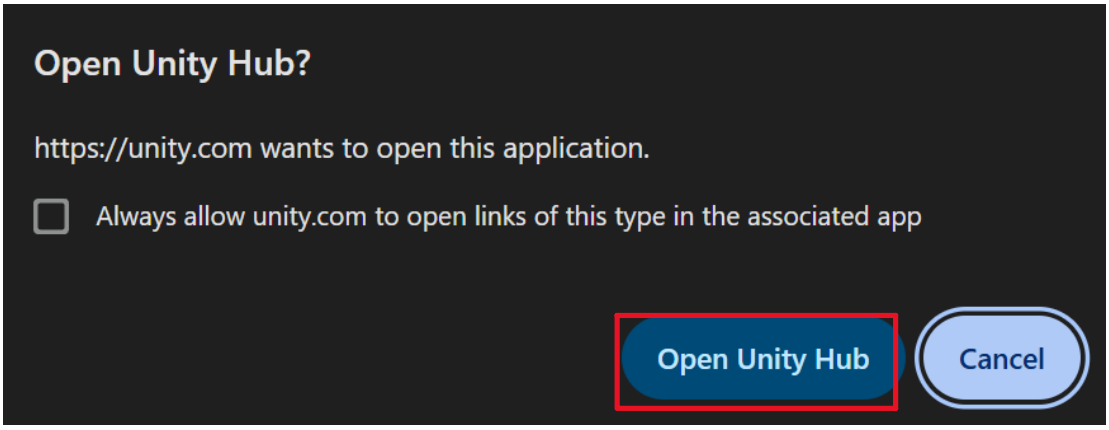

Figure 12 - Unity's Webpage Popup

4. Select the modules you want the version to use, then click install.

Learn

The learn section is one of Unity Hub's new features. In it, you can find beginner courses and tutorials to learn the basics of Unity. Feel free to check out and follow along with any tutorials or courses in the learn section.

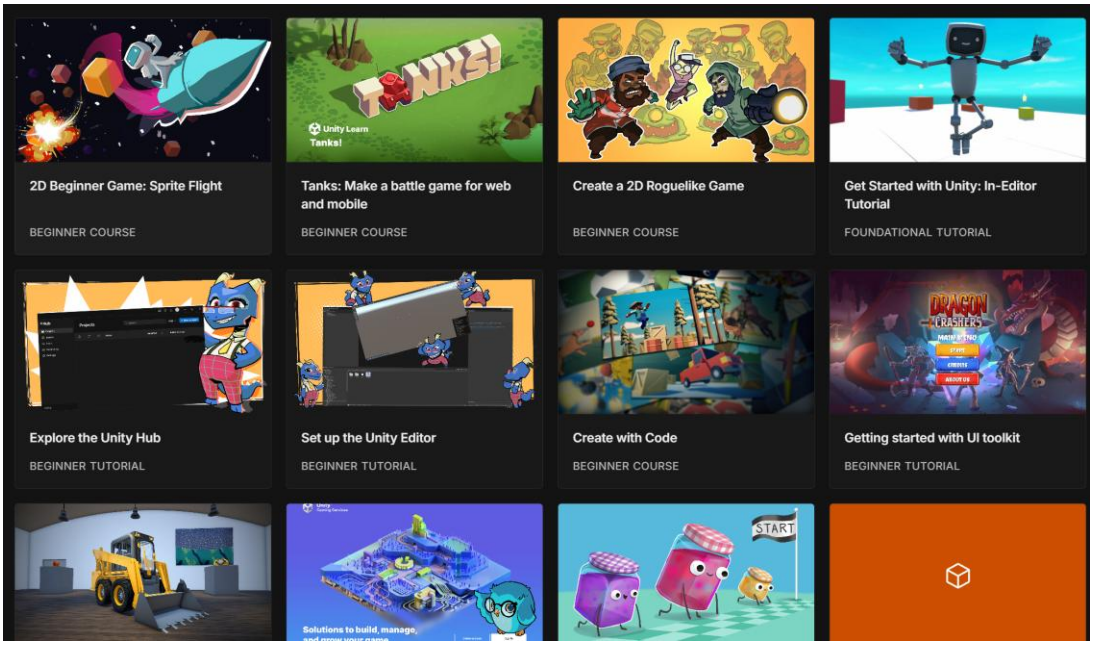

Figure 13 - Sample Content in The Learn Section of Unity Hub

Resources

The resources section is another new feature in Unity Hub. In it, you may find community-based resources and resources designed by Unity itself.

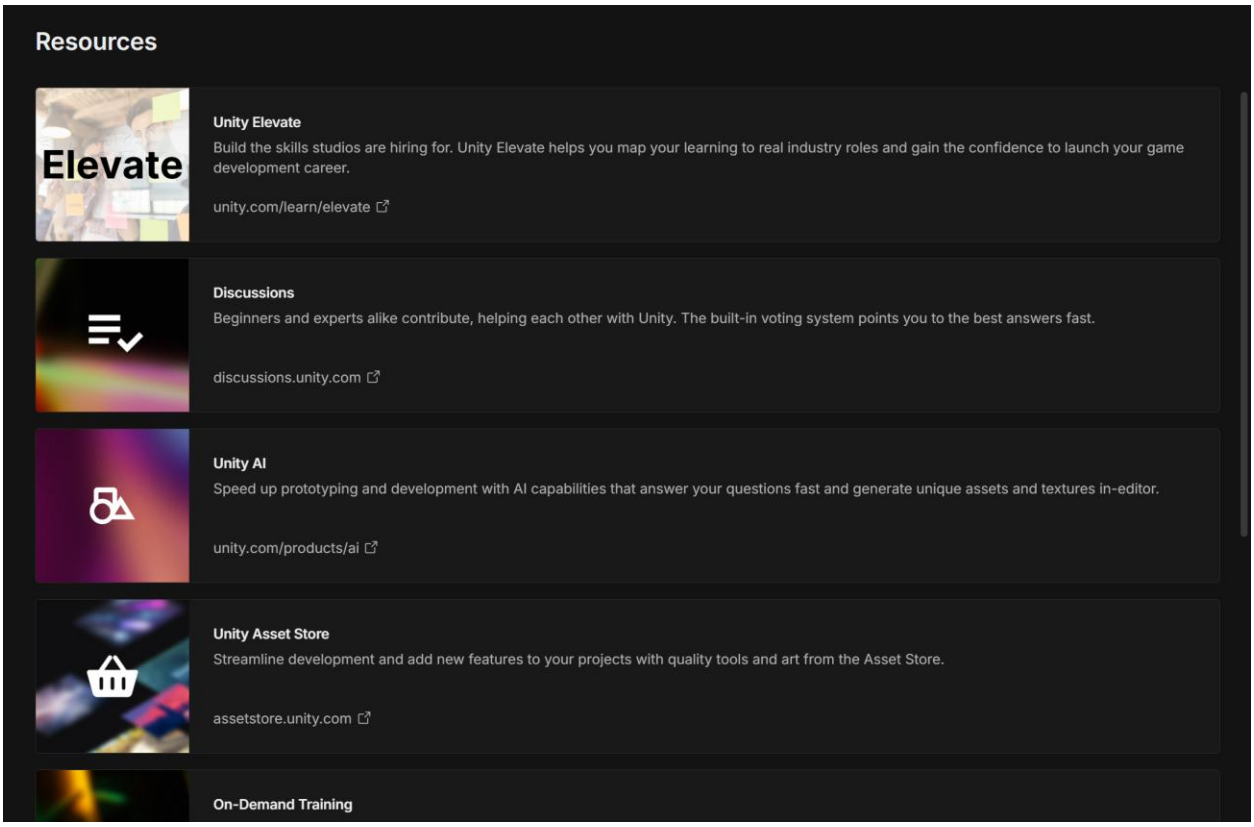

Figure 14 - Sample View of Unity Hub's Resources

The best resource you can find is the asset store. Unity's asset store is primarily community-driven, and anyone can participate. The asset store has practically everything you can think of for video games. You can mainly find 2D and 3D assets for your game. However, you can also find audio and game templates to kickstart your project. Most assets require payment, but there are free resources in the store. Be sure to read the asset licenses before using or distributing your game.

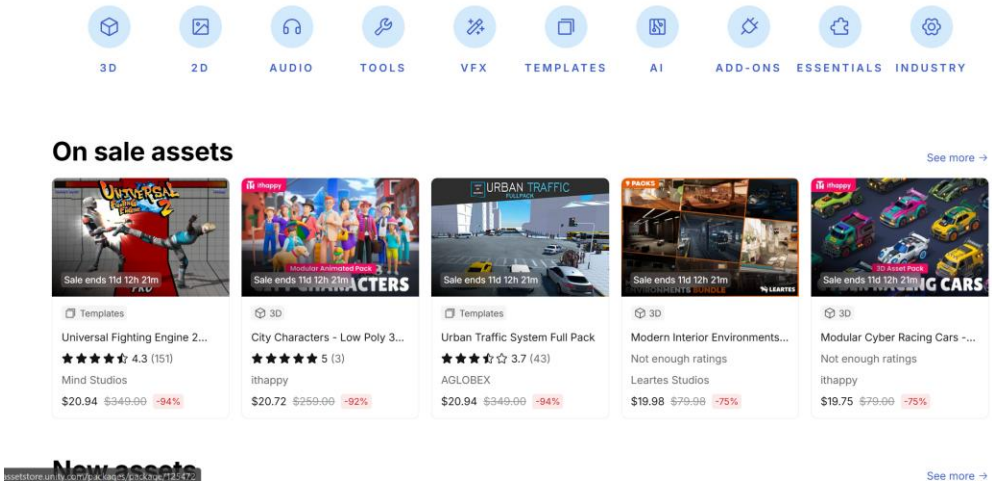

Figure 15 - Unity's Asset Store Landing Page

Licenses

The licenses section lets you control what licenses are activated on your computer. You are most likely using Unity's free license plan. If not, you can add a license in the license section by clicking the blue button in the top-right corner. Select a means of redeeming your license, and Unity will apply it to your computer.

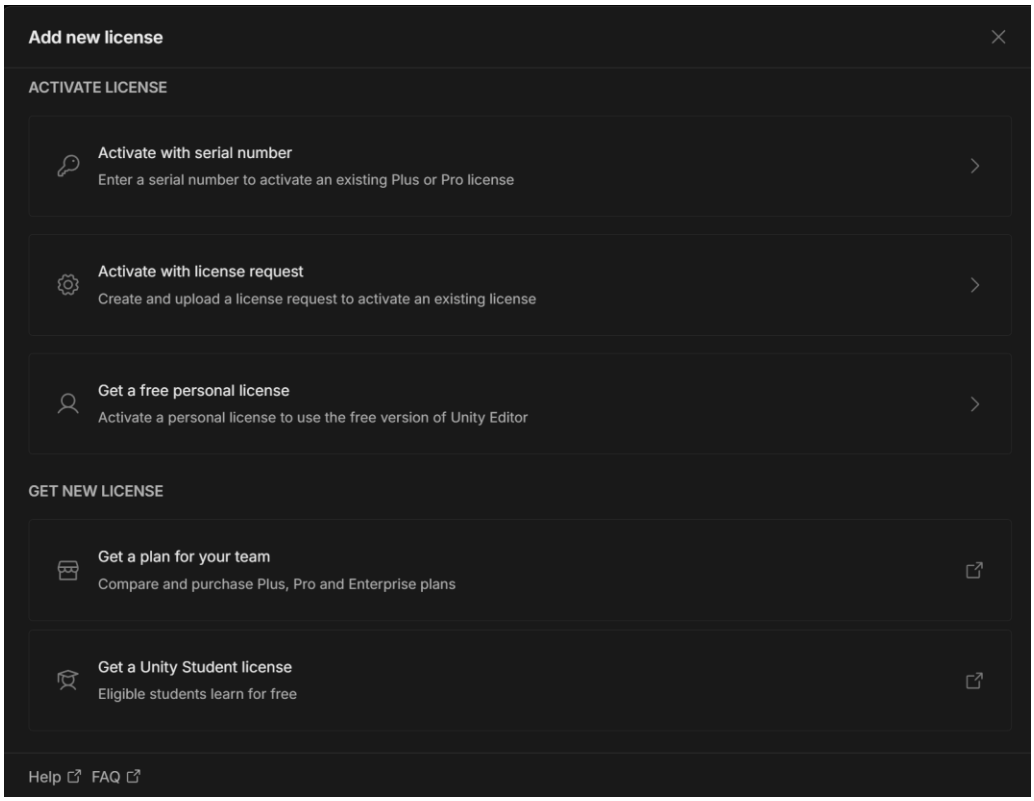

Figure 16 - Sample View of Unity Hub's Licenses Section

The Unity Editor

When opening a new Unity project, you will see many windows. If this is your first time opening Unity, you will see Unity's default layout. The default layout has many windows open at once, including:

- The hierarchy
- Project assets
- The scene view
- The inspector
- The menu bar

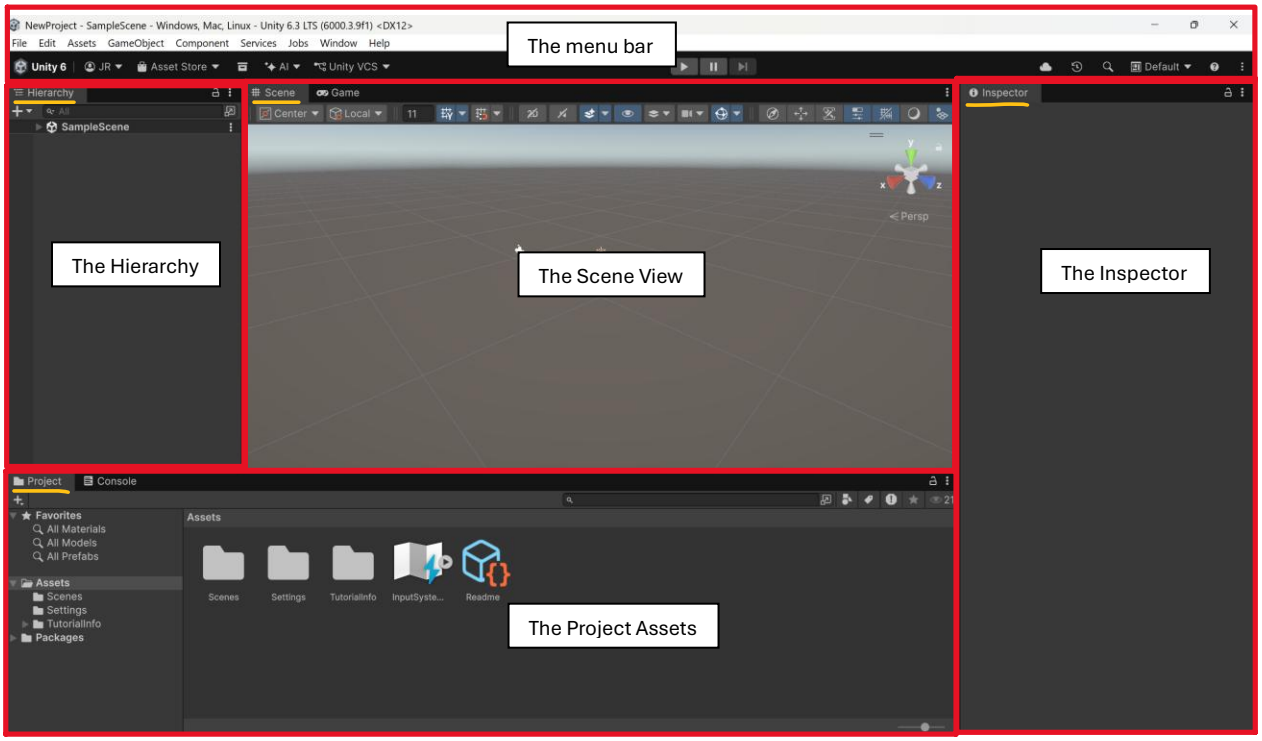

Figure 17 - Default Layout View of The Unity Editor

The Hierarchy

The hierarchy lets you view all your game objects in the open scene. You can see the name of the scene and the game objects within. By default, your scene will have a main camera, a directional light, and a global volume game object.

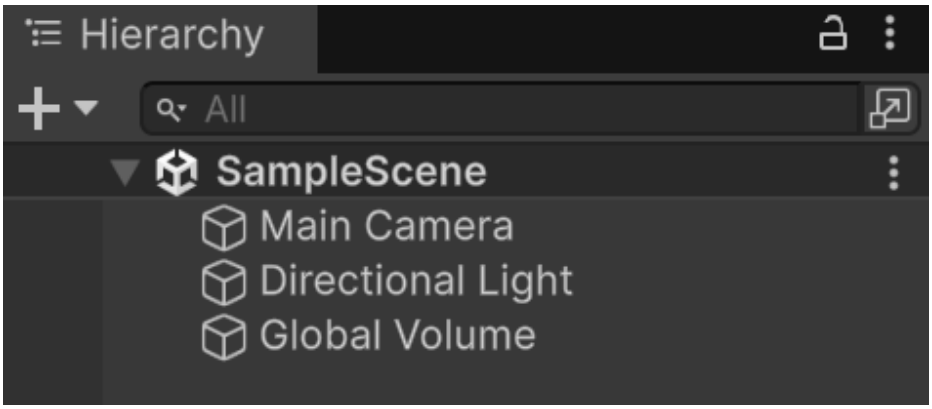

Figure 18 - Starter Hierarchy Setup

You can add your own game objects to the scene. Right-click in the hierarchy's empty space. You will be prompted accordingly:

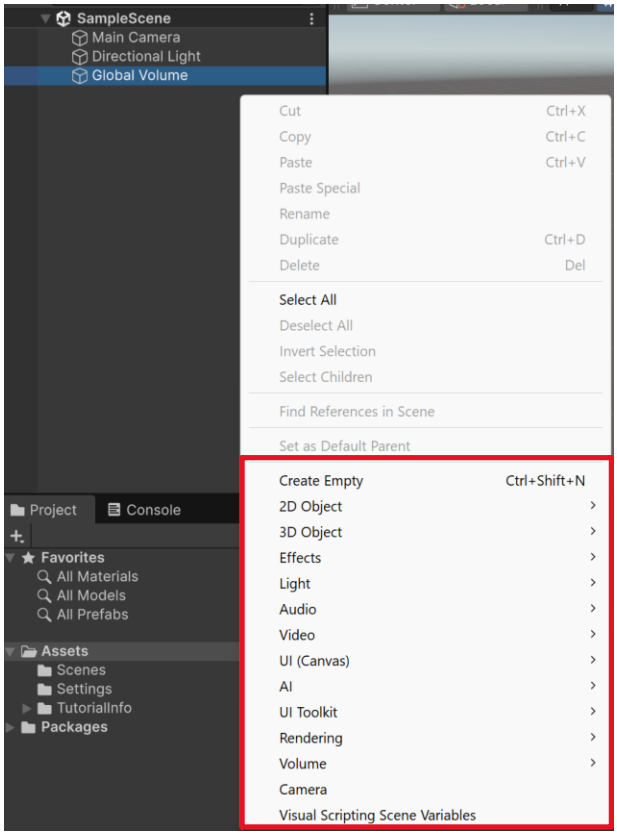

Figure 19 - Location for Where to Add New Game Objects

You can create a game object of your choice. If you create an empty game object, you will add a new game object that has no functionality or visuals. However, the game object will have a transform component, or a position, rotation, and scale in your world space. For primitive shapes, look under the 3D object list.

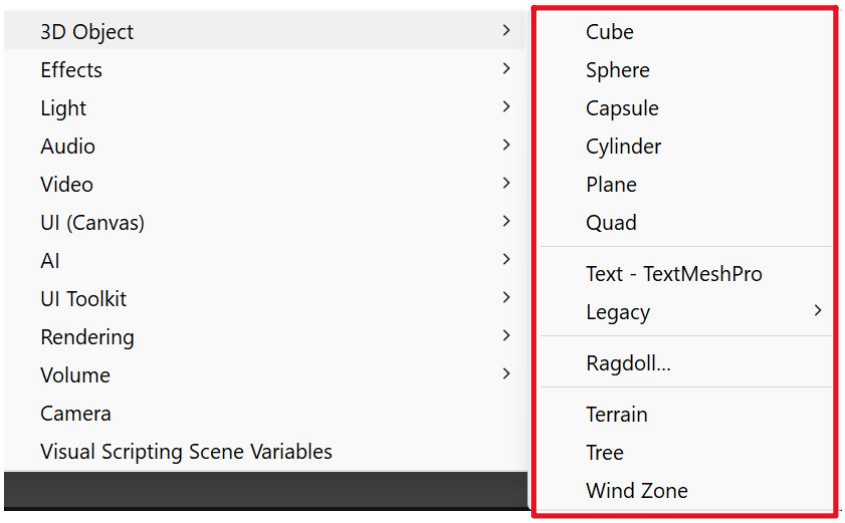

Figure 20 - Location to Add Primitive Game Objects

Try adding a cube to your hierarchy, and give it a name. Upon creation, you should see it in your scene window. Additionally, when you click your cube, you should see all its properties in the inspector window.

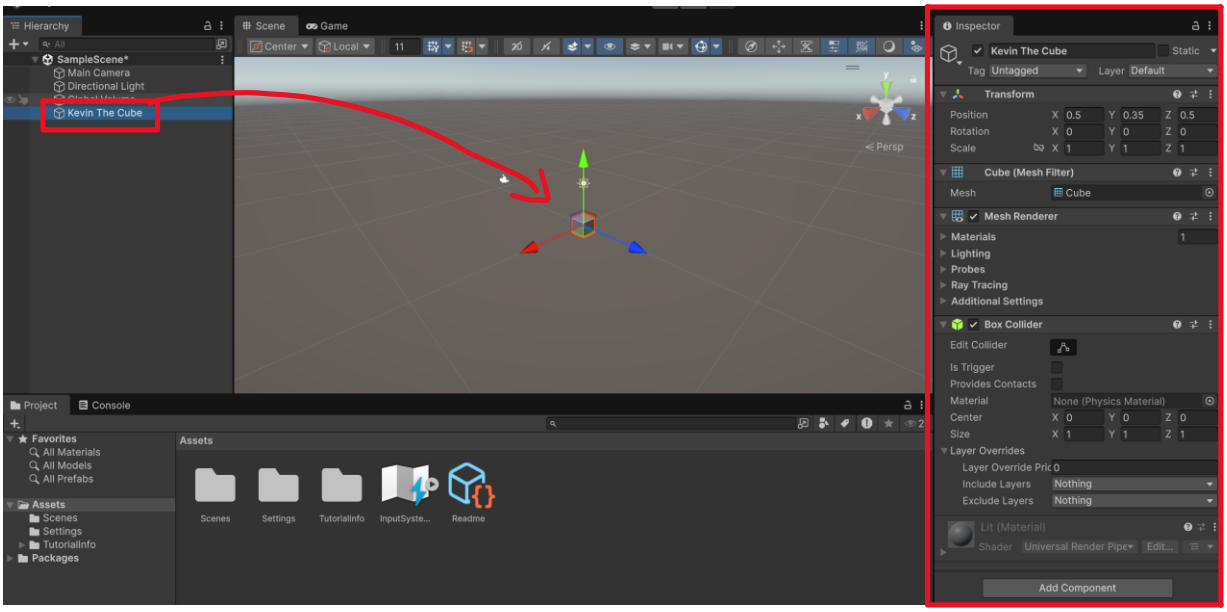

Figure 21 - Inspecting The New Game Object's Properties & Seeing it In Scene View

You can delete game objects from your hierarchy. Click on an object in your hierarchy, and press the delete key on your keyboard. Alternatively, right-click on the object and delete it.

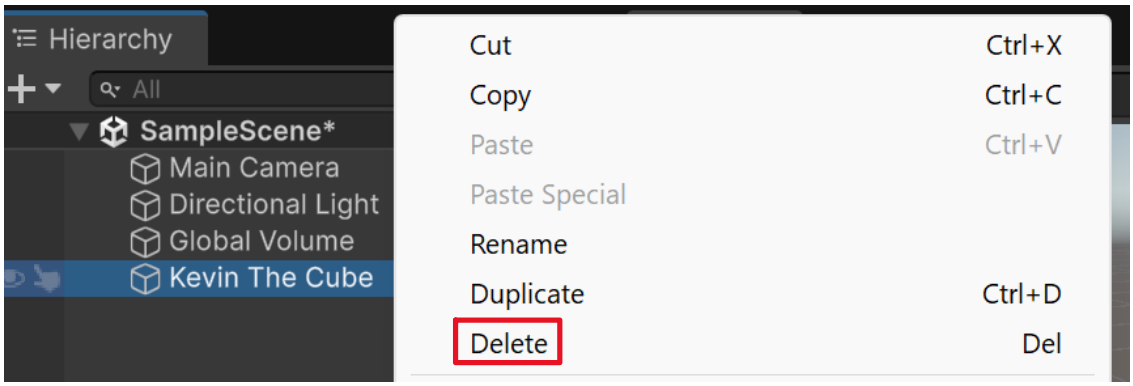

Figure 22 - Deleting Game Objects from The Hierarchy

The Project Assets

The project assets window is where all game assets live in your project. Consider the project assets window as the equivalent to GameMaker Studio's asset browser. However, Unity supports many more assets than GameMaker Studio, including, but not limited to:

- C#/MonoBehaviour and editor scripts
- Materials, shaders, shader graphs, lighting profiles, and other rendering assets
- FBX, OBJ, DAE, BLEND, and other 3D modeling assets
- PNG, JPG, TGA, PSD, and other 2D imaging assets
- WAV, MP3, OGG, and other audio assets
- Animation clips, curves, controllers, and avatar masks
- Scenes, prefabs, input actions, and other Unity-Specific asset types
- Physics materials
- Scriptable objects, JSON files, XML files, text files, and other raw data assets
- Unity packages

Unity allows you to create some assets in the engine itself. However, many assets are best created outside of Unity and imported as a proper file type from specialized software. For example, since Unity does not support 3D modeling, you will commonly create 3D models in software like Blender and export them as FBX files into Unity.

Creating Assets in Unity

To create an asset in Unity, right-click in the empty space in the project assets window. Click " Create " at the top of the pop-up. Unity lists many types of assets you can customize in the editor, with common ones at the top, such as C#/MonoBehaviour scripts and materials.

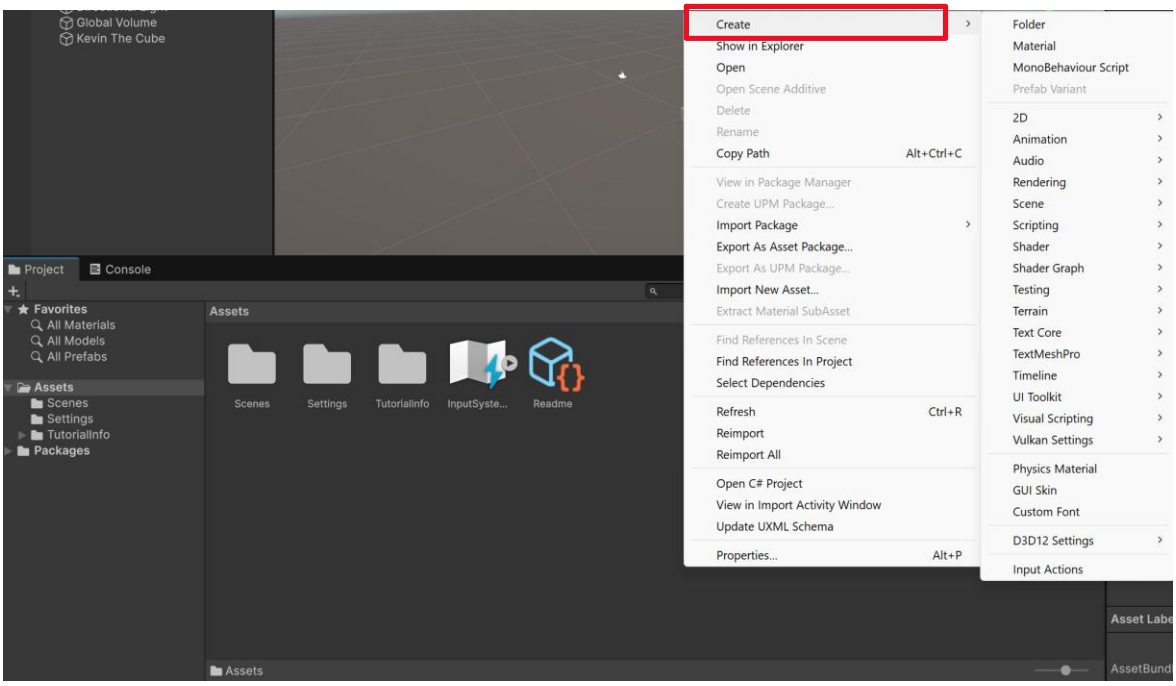

Figure 23 - Location to Create New Assets In Unity's Project Window

Let's look at two common assets: materials and MonoBehaviour scripts.

1. Create a material, and name it a color of your choice.

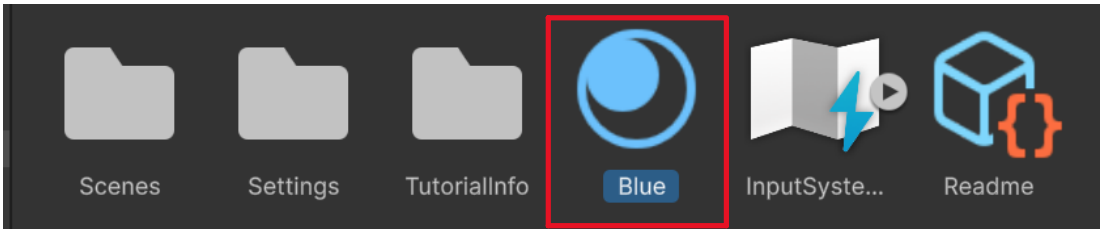

Figure 24 - Viewing The New Material in The Project Window

2. Left-Click your new material asset, and view its properties in the inspector window.
    a. Note: There's a lot to cover with materials. For now, you just need to know that the base map controls the color of your material.

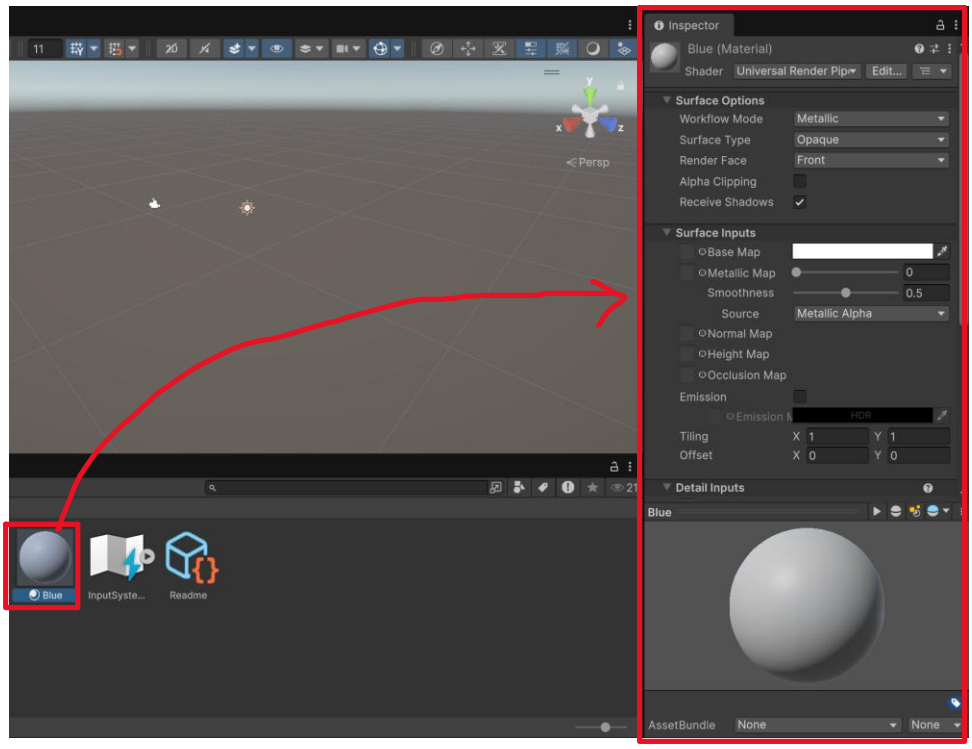

Figure 25 - Viewing The Material's Properties in The Inspector

3. Click on the base map's color to open the color map. Then, use the color wheel to set your material's color to the name you gave it.

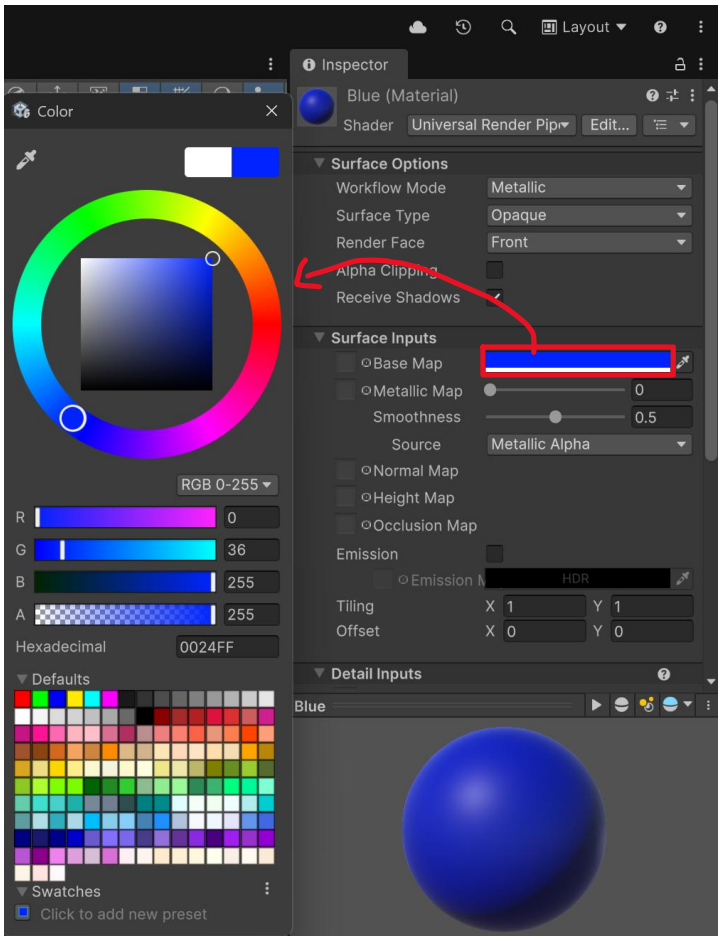

Figure 26 - Changing The Material's Base Color

4. You can now use the material on an object in your game. If you do not have a visible 3D object in your game, create one. Then, drag and drop your material onto your visible object **in the scene view.**

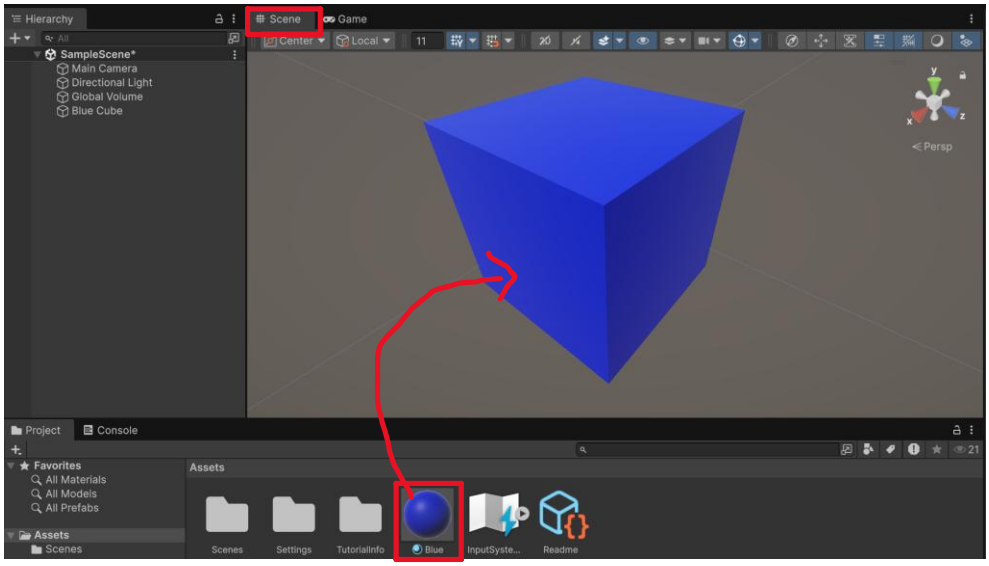

Figure 27 - Applying The Material To a Game Object

This is a simple way to customize Unity's primitive shapes with colors. However, you cannot fully customize materials in Unity without using texture files or Unity's shader graphs. To learn more about shading and rendering, I recommend learning Blender and checking out the documentation on importing Blender assets into various game engines.

Let's look at creating a MonoBehaviour script:

1. In the empty space of the project assets window, right-click and create a new MonoBehaviour script. Name it "DisplayMessage".

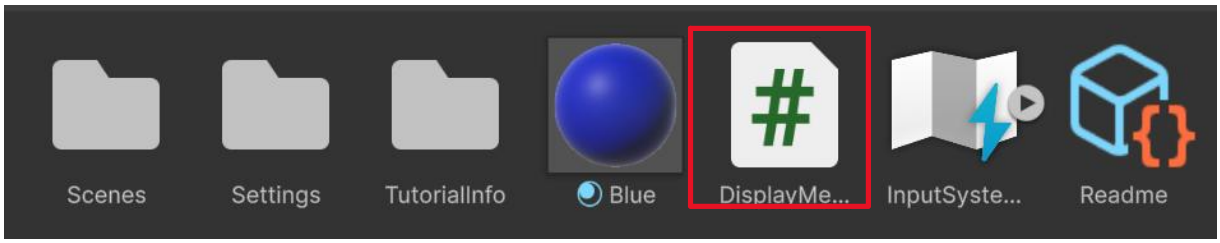

Figure 28 - Creating a New C#/MonoBehaviour Script

2. Double-click your script to open it in Visual Studio. Then, type the following in your script:

```
using UnityEngine;

Unity Script | 0 references
public class DisplayMessage : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    Unity Message | 0 references
    void Start()
    {
        Debug.Log("Hello World!");
    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {

    }
}
```

Figure 29 - Basic Setup of New Script

3. Save your script (CTRL+s or File->Save All) and return to the Unity editor. It may take a moment for Unity to reload its domain.

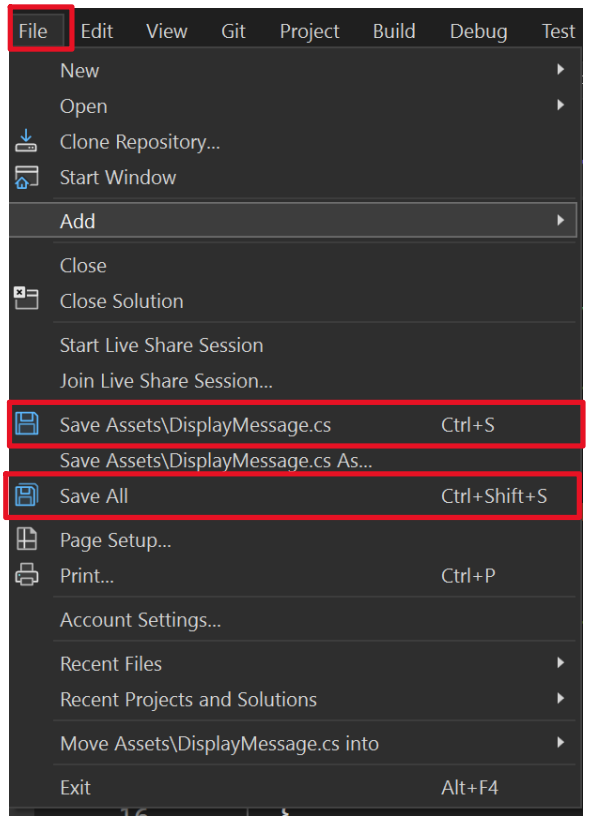

Figure 30 - Saving The Script in The IDE

Note: Even though the script's functionality was created, **it will not run unless it is attached as a component on a game object in your scene**. Thus, we need to create a

game object in the scene, attach our script as a component, and then run our game to see the results.

4. Create an empty game object in your hierarchy and name it “Display Message”.

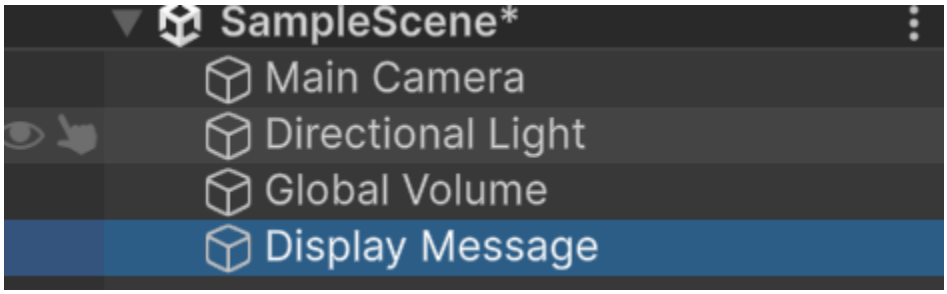

Figure 31 - Creating a New Game Object to Hold The Script

5. Left-click your object in the hierarchy, and view its properties. Click the “Add Component” button at the bottom, and use the search bar to find your DisplayMessage script.

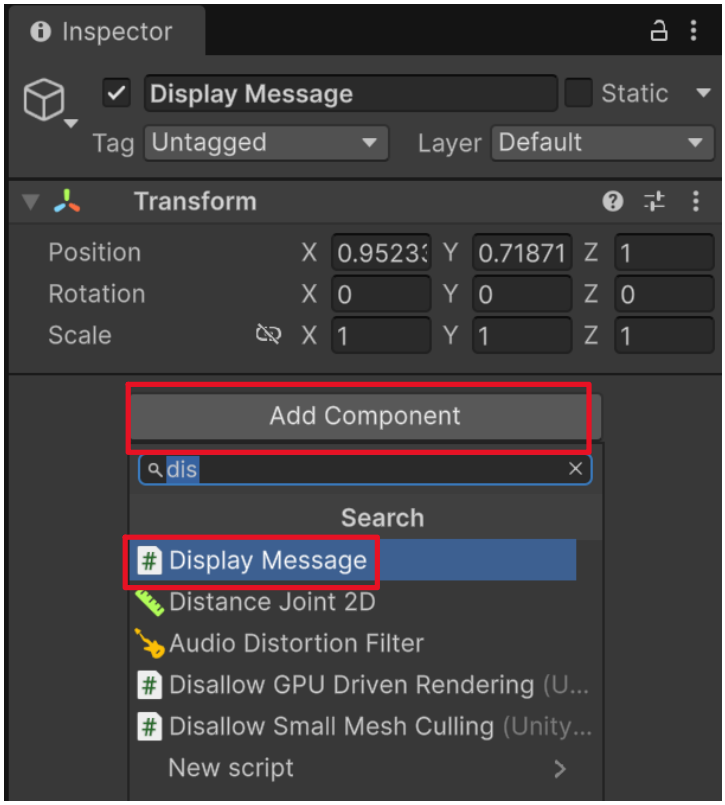

Figure 32 - Adding Our Script as a Component to The New Game Object

6. (alternative to Step 5) While you have your game object’s properties shown in the inspector, drag and drop your DisplayMessage script asset from the project asset

window into the empty space in the object's inspector. This is a quick way to add assets as components to game objects.

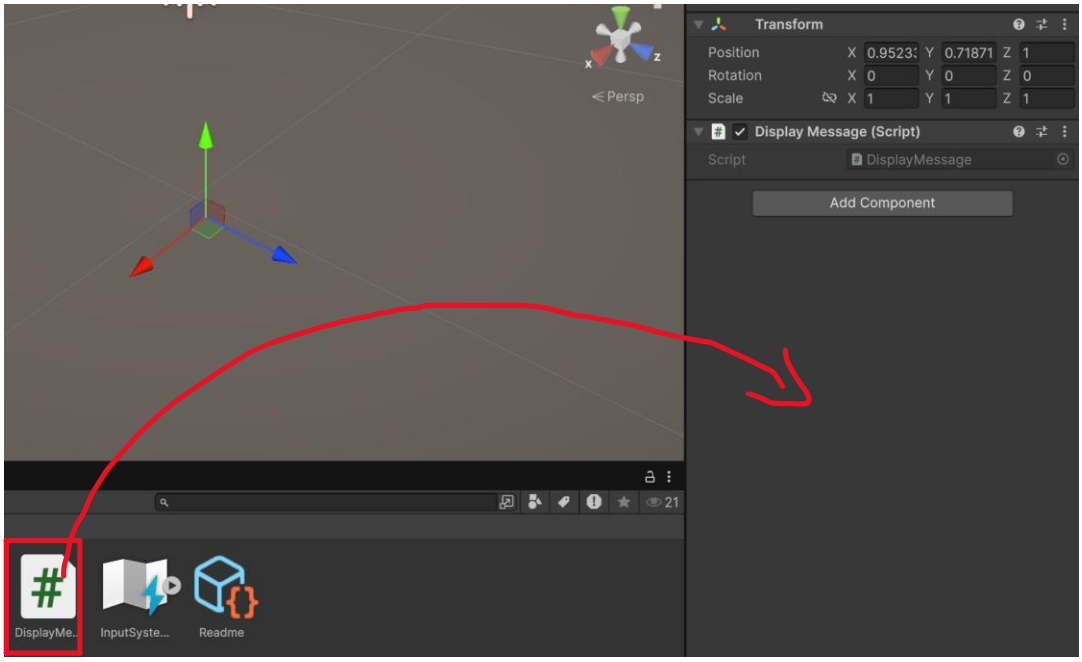

Figure 33 - Quickly Adding The Script to Our Game Object

7. Click the play button at the top of the game. This will open your game window.

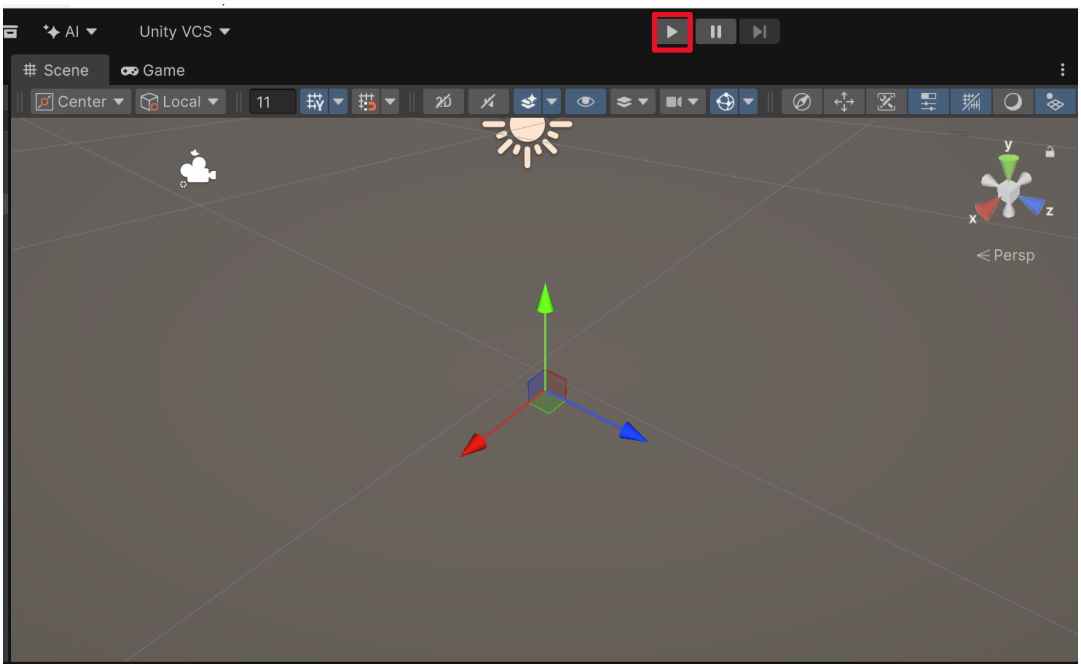

Figure 34 - Location of The Play Button to Run The Game

8. You can view the message in the console window. By default, Unity has the console window docked next to the project asset window at the bottom of the editor. Open the console window while your game is running. You should see the message:

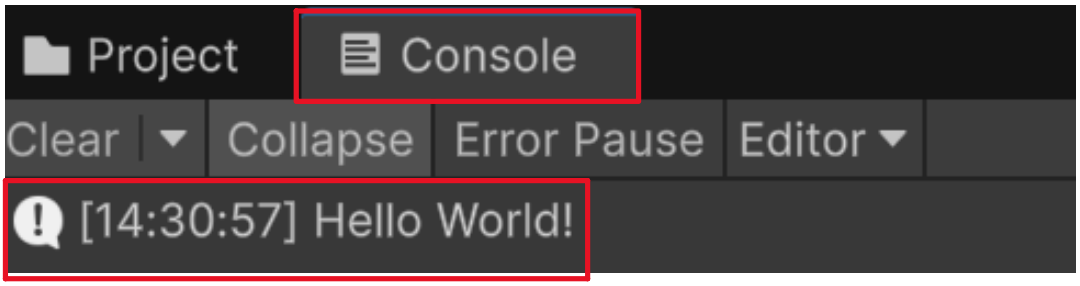

Figure 35 - Sample Output from Our Script Attached to The Game Object

Organizing Assets in The Project Assets Window

Asset organization can save you countless hours of searching for a specific asset. While Unity has a search function to find assets by name or type, you will commonly forget many assets that you added to your project.

The best solution to organize assets in Unity is to group assets in folders. Then, you can create a folder hierarchy to help you find assets in your project. Below, you may find three methodologies to organize your assets:

Organization Methodology #1: Grouping By Asset Type

Under this methodology, assets in your project are grouped based on the asset's file type. For example, you would create a folder called scripts, materials, prefabs, etc., and move your assets to their respective folders.

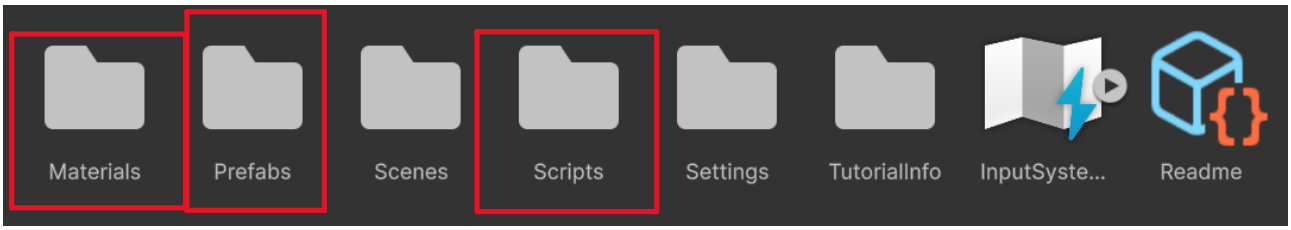

Figure 36 - Grouping Assets By Their Type

You can take this methodology one step further. After grouping assets by their file type, you can group them by their purpose. For example, say you have a player game object in your game. Under your scripts folder, you would create a player subfolder and place all player-related scripts in that folder.

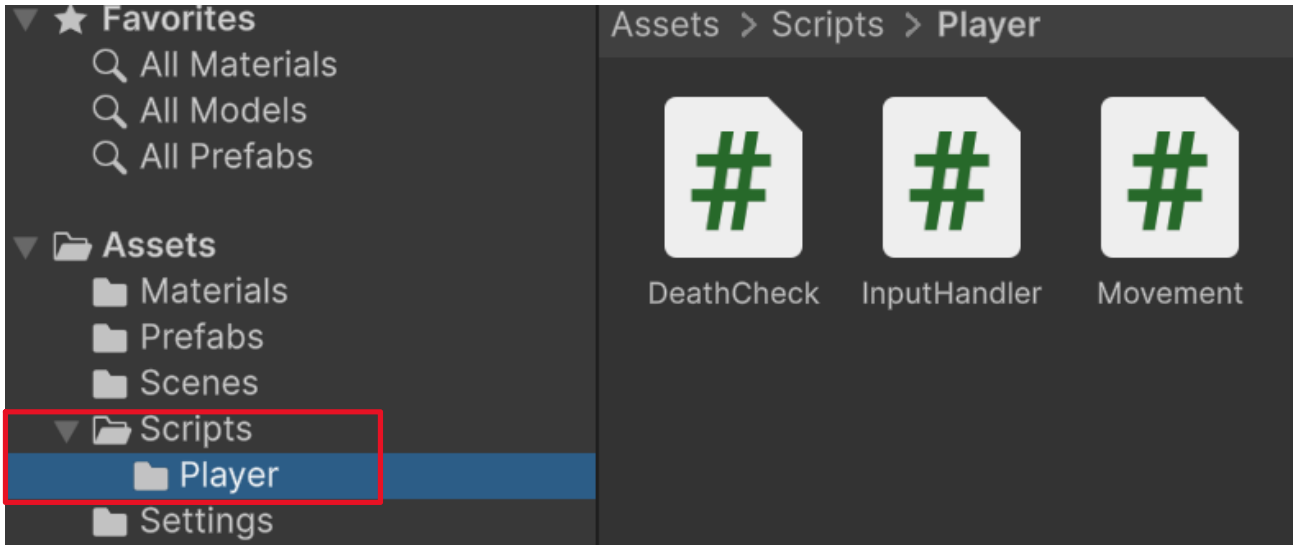

Figure 37 - Further Grouping Asset Types by Their Related Purpose

You can continue this folder hierarchy across your project, and it's a great way to start organizing your assets for small projects.

Organizational Methodology #2: Grouping By Related Purpose

This methodology groups assets into folders designated for a specific purpose. For example, say you have a player game object. A player is composed of multiple asset types, including scripts, models, materials, animations, and more. Under this methodology, all player-related assets are grouped in a folder called player.

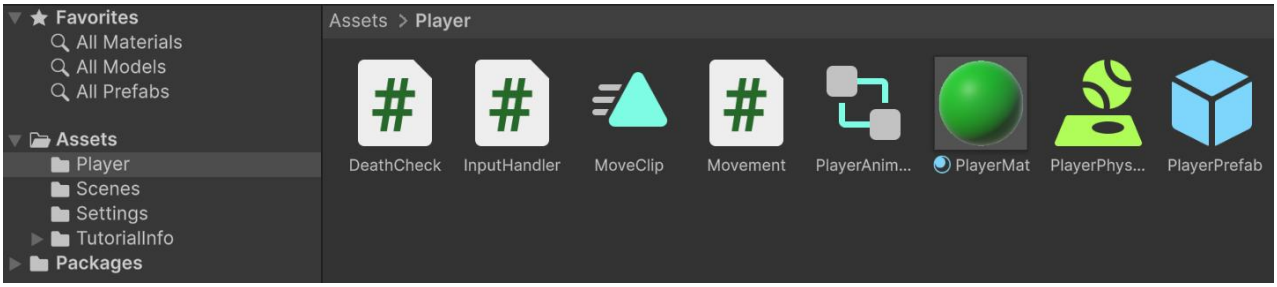

Figure 38 - Grouping Assets by Their Related Purpose

After grouping assets by their related purpose, you can then apply *Methodology #1: Grouping By Asset Type* to organize your assets even more. In the end, you will end up with a hierarchical structure in which related assets are grouped by their purpose, then separated by asset type.

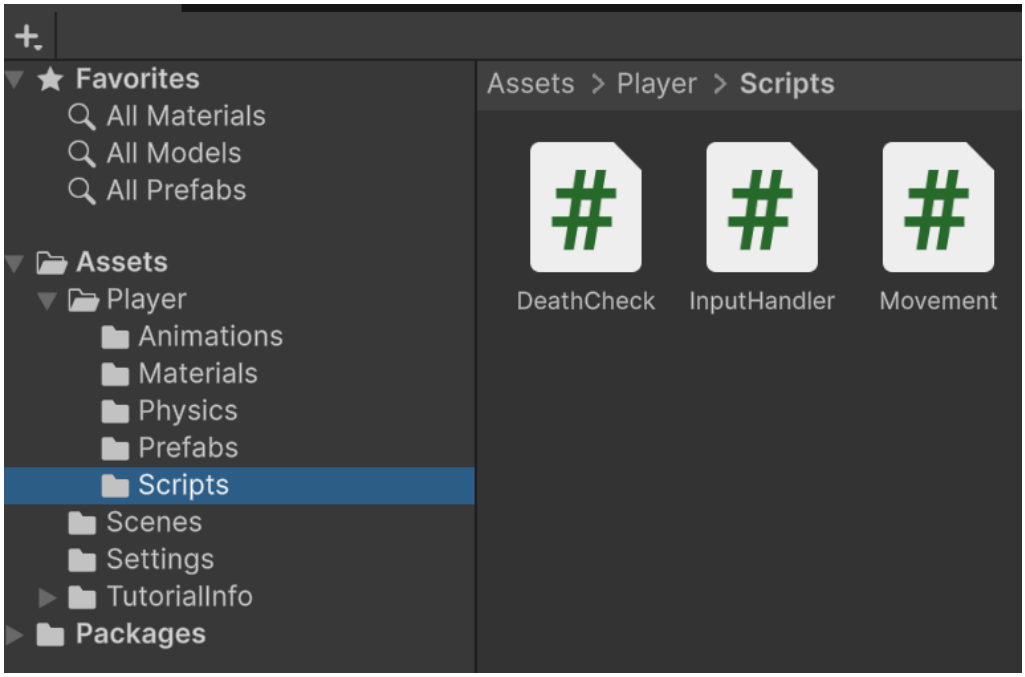

Figure 39 - Further Grouping Related Assets By Their Type

Organization Methodology #3: Grouping By Organizational Teams

This methodology breaks assets up into groups based on the teams working on the project, and it is common practice when working with many specialized teams. For example, say you are part of a large game project. You have a team specifically for artwork, programming, audio, and game design. You would create a folder for each team. Then, you can break down your assets by the team that owns them, and apply methodology #2 to group related assets accordingly.

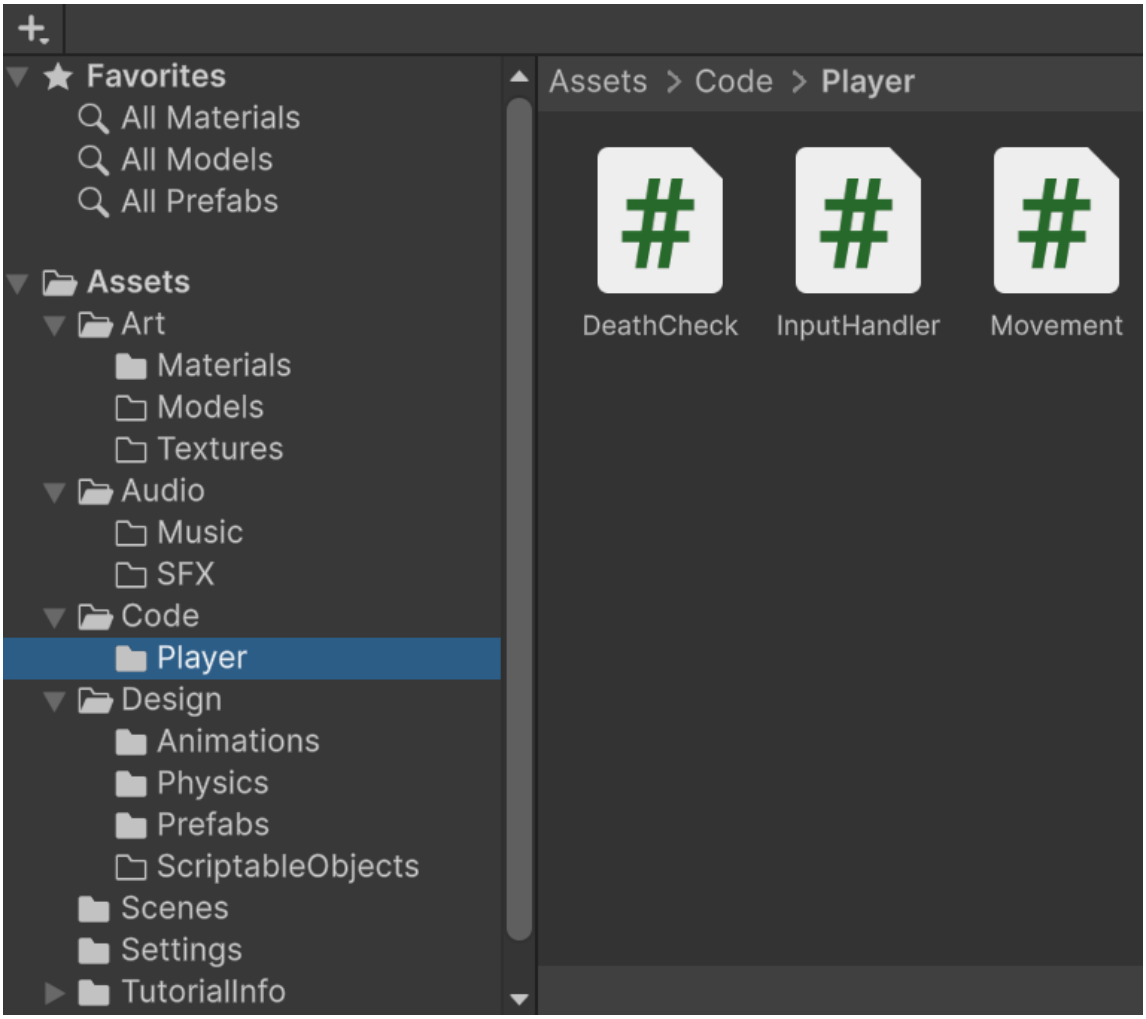

Figure 40 - Grouping Assets By The Organizational Teams

The Scene View

The scene view shows you the scene's world and every game object created in it. Unity's scenes are the equivalent of GameMaker Studio's rooms. Scenes hold game objects and their respective components. Scenes then use the objects and components to communicate with Unity's game engine for rendering, physics, sound, and more. Additionally, scenes are where your game is played. Thus, anything the player does or sees must exist in a scene.

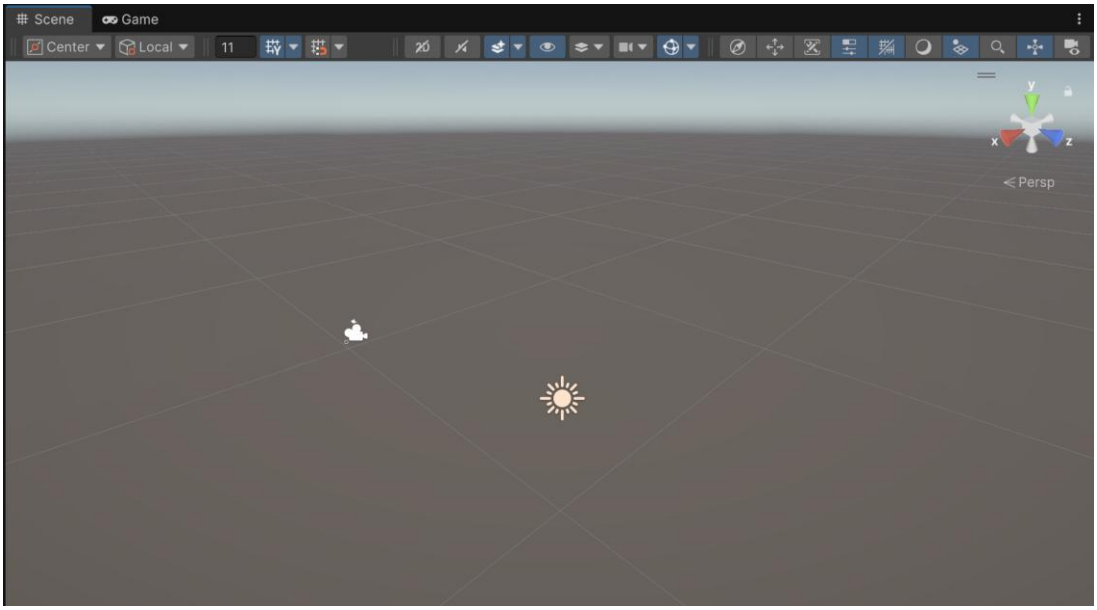

Figure 41 - Starter Scene View

Scenes are broken down into many parts, and there's a lot to each part. Essentially, Unity's scenes can be broken down into visual debugging, object transformation, scene navigation, and settings.

Scene Navigation

Unity provides basic input controls for scenes. This includes the following:

- F (while a game object is selected): Center the object on the camera's screen, and move the camera close to it
- H (while a game object is selected): Hide the game object in scene view.
- L (while a game object is selected): Toggles the object's scene pickability; when on, you cannot select the object from scene view, and vice versa.

Additionally, Unity provides many navigation controls to help you navigate scenes. Scene navigation can be broken down into 2D and 3D spaces, since the navigation controls operate differently based on the scene view. Also, you can view your scene from two different camera perspectives: perspective and isometric. Let's break this down:

2D Scene Navigation

Navigating your scene in 2D is done through up/down/left/right movement. You can do this by either holding the right-click or the middle mouse button and dragging your cursor in the opposite direction you want to move.

3D Scene Navigation

Unity provides many ways to navigate 3D scenes. Let's break down all the controls:

- Right-Click (hold): allows you to rotate the camera in the direction you move the cursor
- Right-Click + E (hold): allows you to move the camera upward, relative to its looking direction
- Right-Click + Q (hold): allows you to move the camera downward, relative to its looking direction
- Right-Click + W (hold): allows you to move the camera forward, relative to its looking direction
- Right-Click + S (hold): allows you to move the camera backward, relative to its looking direction
- Right-Click + A (hold): allows you to move the camera left, relative to its looking direction
- Right-Click + D (hold): allows you to move the camera right, relative to its looking direction
- Mouse Scroll Forward: moves the camera forward, relative to its looking direction
- Mouse Scroll Backward: moves the camera backward, relative to its looking direction
- Right-Click + [Binding] + Shift: Moves the camera in the binding's direction at an accelerating speed
    - Note: The binding can be any of the previous keyboard inputs listed above (Q, W, E, A, S, D)
- Right-Click + Mouse Scroll: Allows you to control how fast the camera accelerates when performing the previous binding

Perspective Camera View

The perspective camera perspective is how you'd expect a camera to see your environment. The camera views the environment as if it were your own eyes.

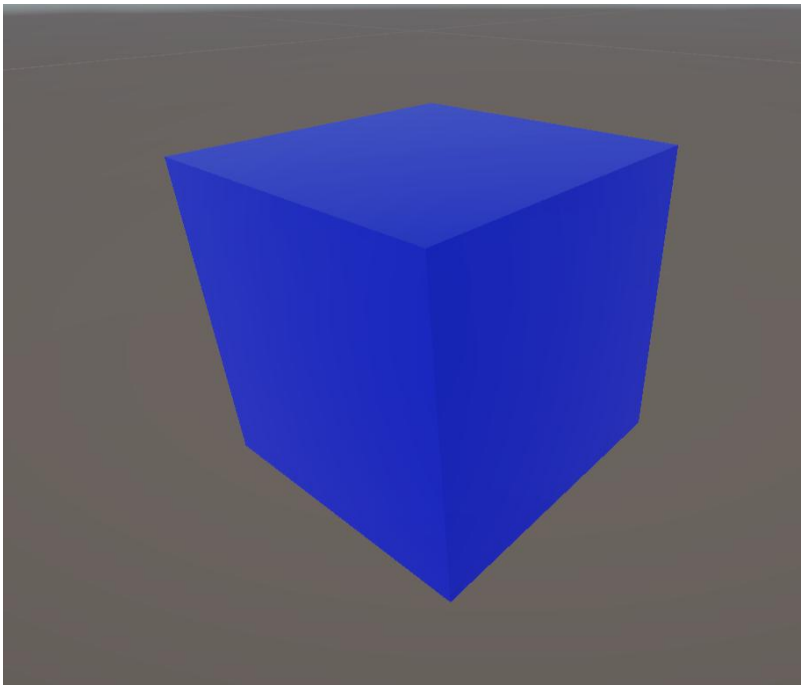

Figure 42 - Perspective/Orthographic View of The Scene

A perspective camera will make objects appear smaller the farther away they are. Additionally, straight lines tend to bend inward of our vision the farther out they go, approaching a vanishing point/horizon plane.

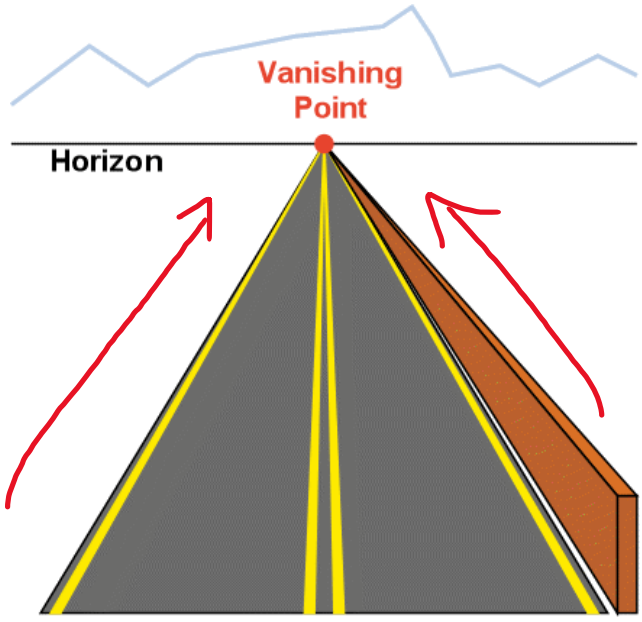

Figure 43 - Explanation for Perspective/Orthographic Views

Unity will default the camera to perspective view. If you accidentally switched to isometric view, you can return to perspective by clicking the “iso” text below the axis orientation.

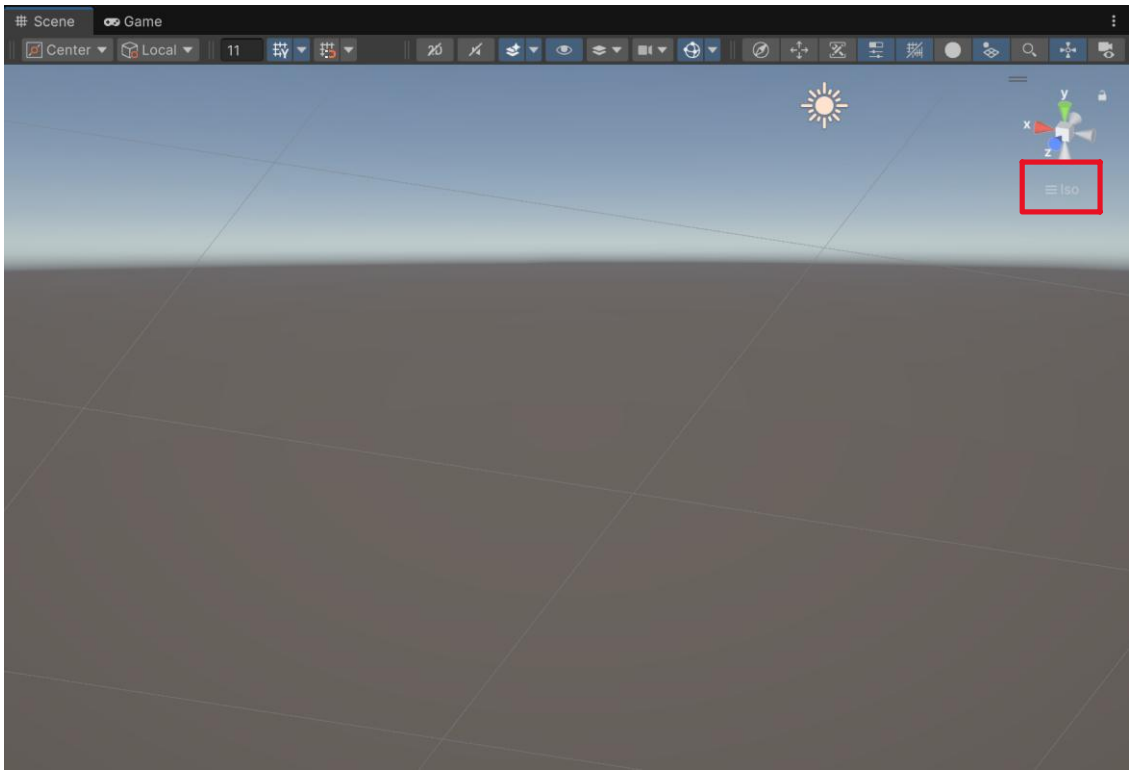

Figure 44 - Location to Swap Your Project Back to Perspective View

Isometric Camera View

An isometric camera removes the distortion generated from perspective. As a result, isometric cameras have little to no depth of field, distant objects appear the same size as close objects, and objects do not feel real.

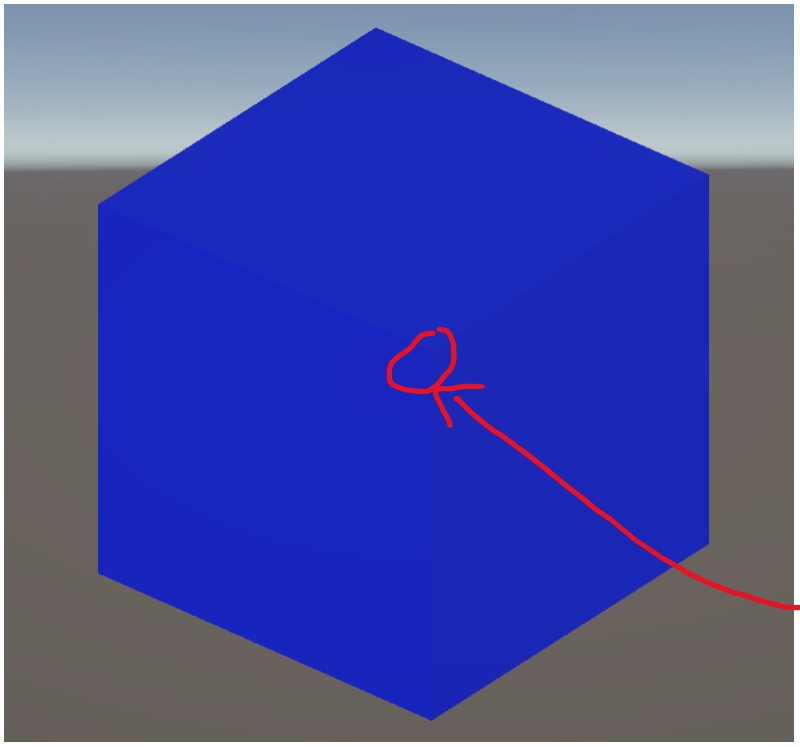

Figure 45 - Isometric View of Scenes

This sounds like a worse camera perspective to have, but isometric cameras are great when you want to work with objects that are distant from one another, and you want to remove distortion.

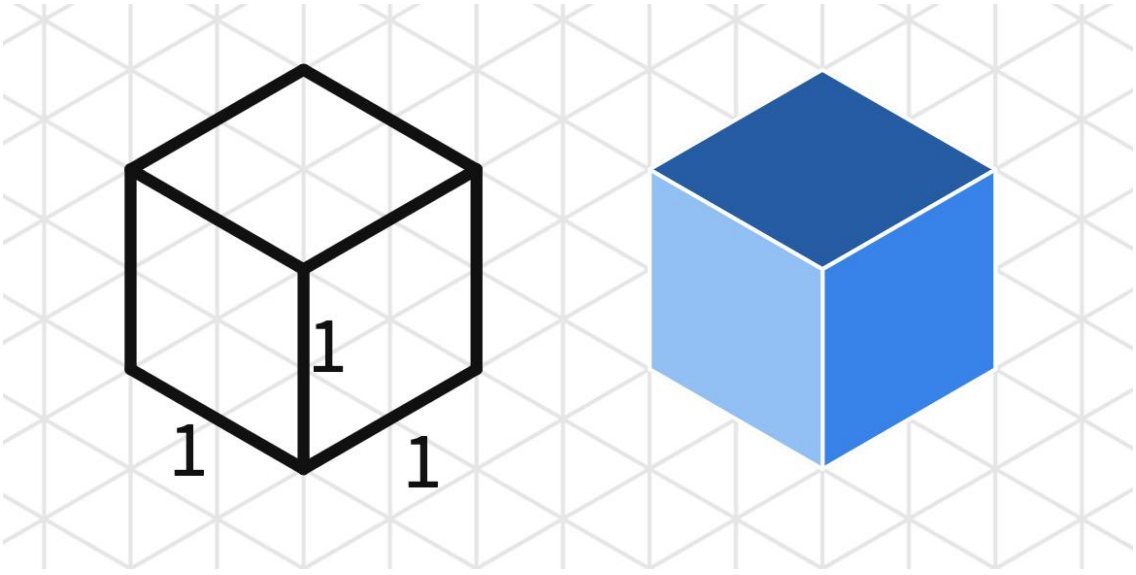

Figure 46 - Supportive Image Showing Isometric Camera Views

Object Transformations

Unity offers many controls for transforming game objects, typically done in the scene view. To start, you will want to have a primitive 3D object ready in your scene. When selecting your game object in the scene, you can perform the following transformation given the corresponding input:

- W: Opens the option to move the object up/down, left/right, and forward/backward.

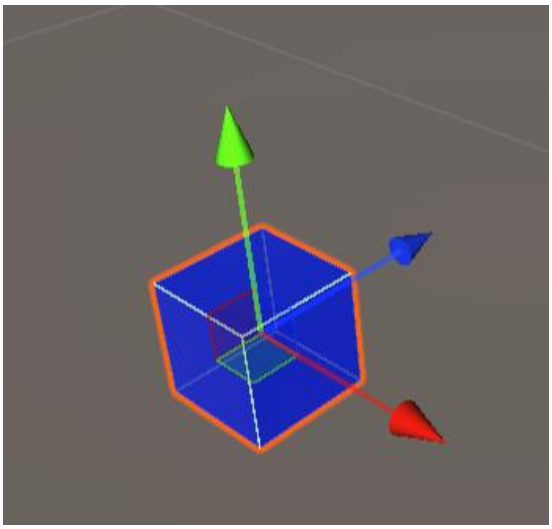

Figure 47 - GameObject Visual When Moving The Object

- E: Opens the option to rotate the object based on its pitch, yaw, and roll.

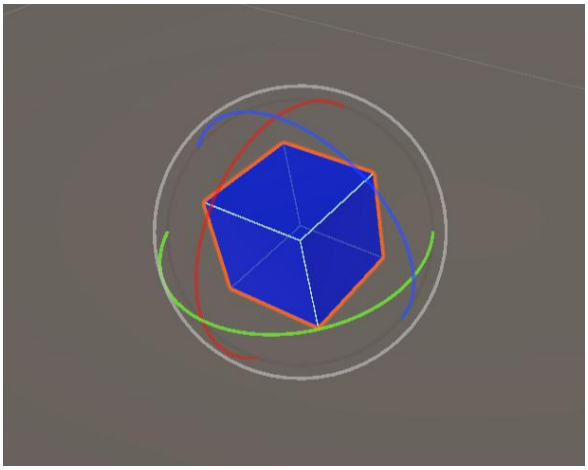

Figure 48 - GameObject Visual When Rotating The Object

- R: Opens the options to scale the object on the x-axis, y-axis, and z-axis. Also, you can grab the inner grey box to scale the object uniformly along the x, y, and z axes.

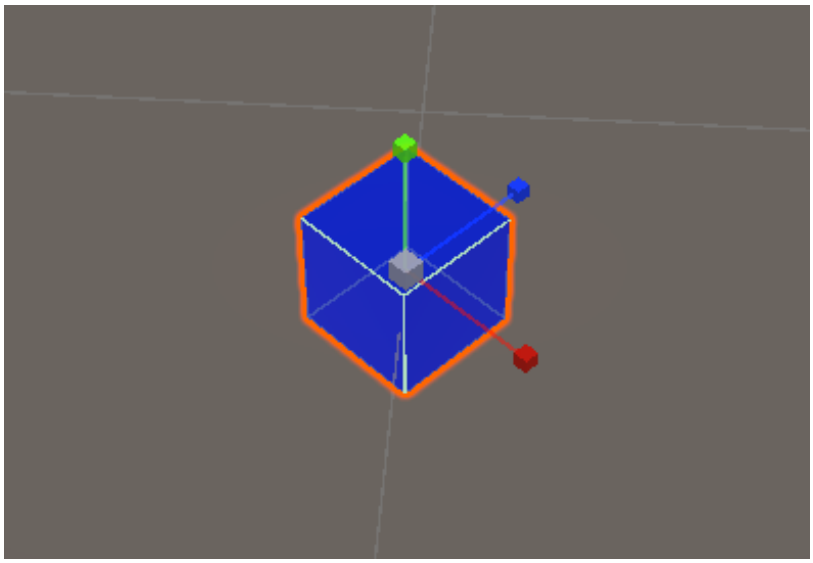

Figure 49 - GameObject Visual When Scaling The Object

- T: Opens the option to stretch the object in a given direction. The directions change depending on how you view the object (from the side or from above).

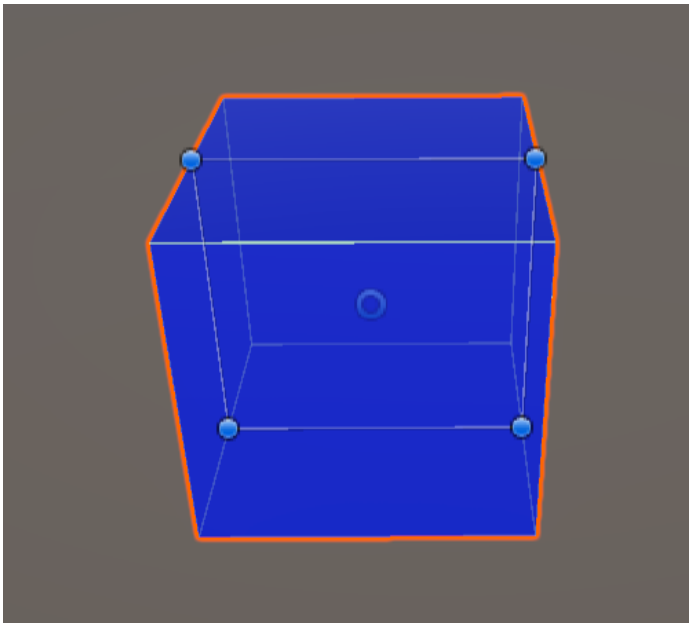

Figure 50 - GameObject Visual When Stretching The Object

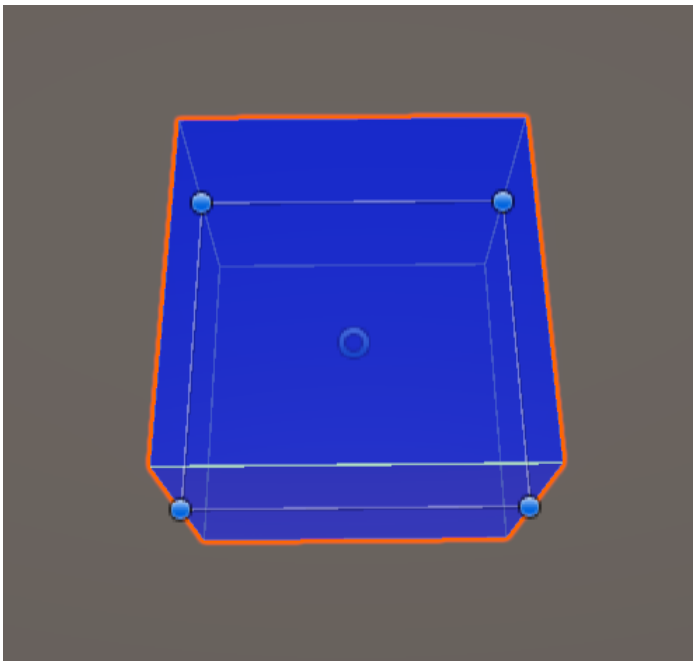

Figure 51 - Upper View of Scaling the Visual (The Gizmos Flips Horizontally)

- Y: Gives you the options to move, rotate, and scale the object, all visible at the same time.

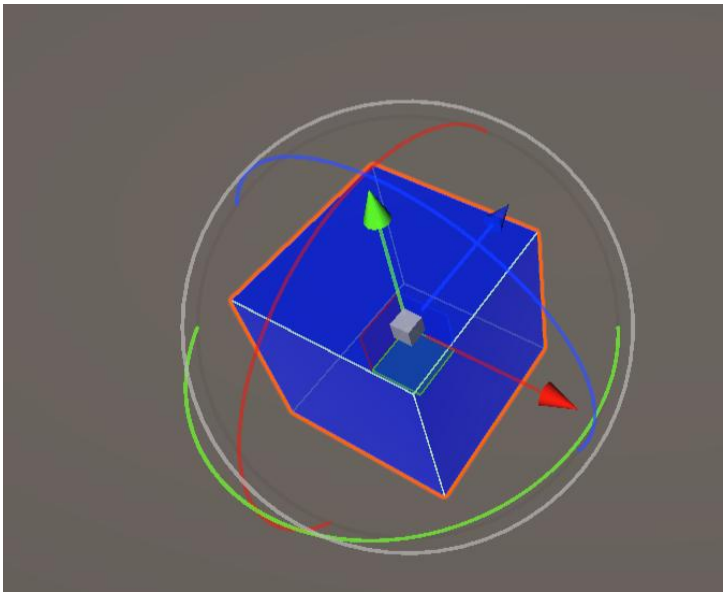

Figure 52 - GameObject Visual When Pressing Y

A Quick Note About Object Pivot & Orientation Settings

The way your object is transformed depends on your scene's pivot and orientation settings. You can find these settings in the Scene ribbon at the top left. Additionally, you can use Z to

toggle between center and pivot mode, and you can use X to toggle between global and local mode.

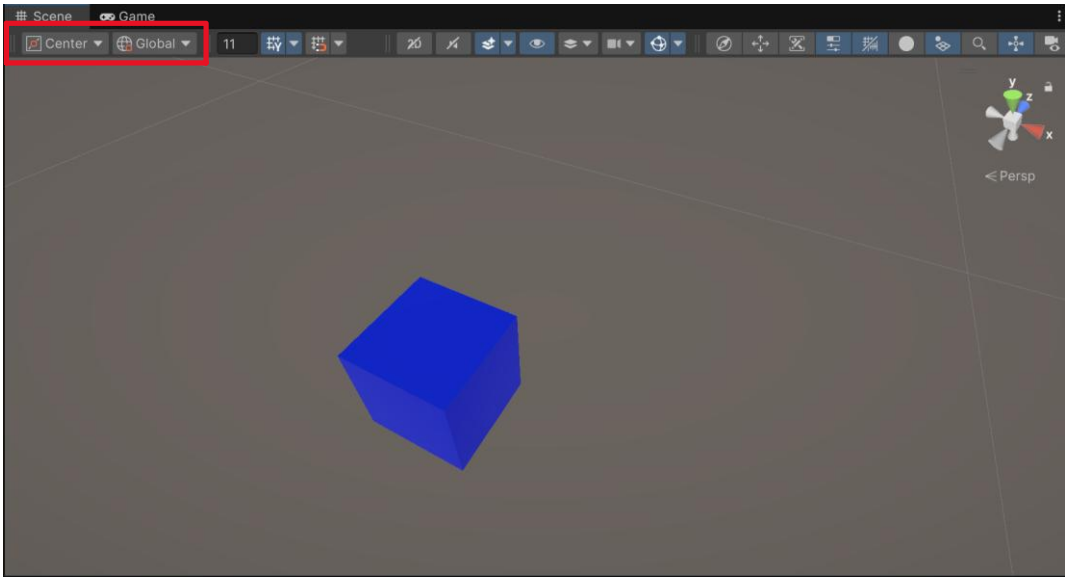

Figure 53 - Location of Pivot and Orientation Settings

There are two pivot and orientation modes: center and pivot for pivot settings, and local and global for orientation settings:

- Center Mode: movement, rotation, and scaling are done from the center of the object
- Pivot Mode: movement, rotation, and scaling are done from the game object's pivot position
- Local Mode: transformations are performed relative to the object's orientation in space
    - E.g., moving an object to the right moves it rightward relative to the direction the object is "looking"
- Global Mode: transformations are performed relative to the world space (x, y, and z-axes)
    - E.g., moving an object to the right moves it 1 unit on the x-axis

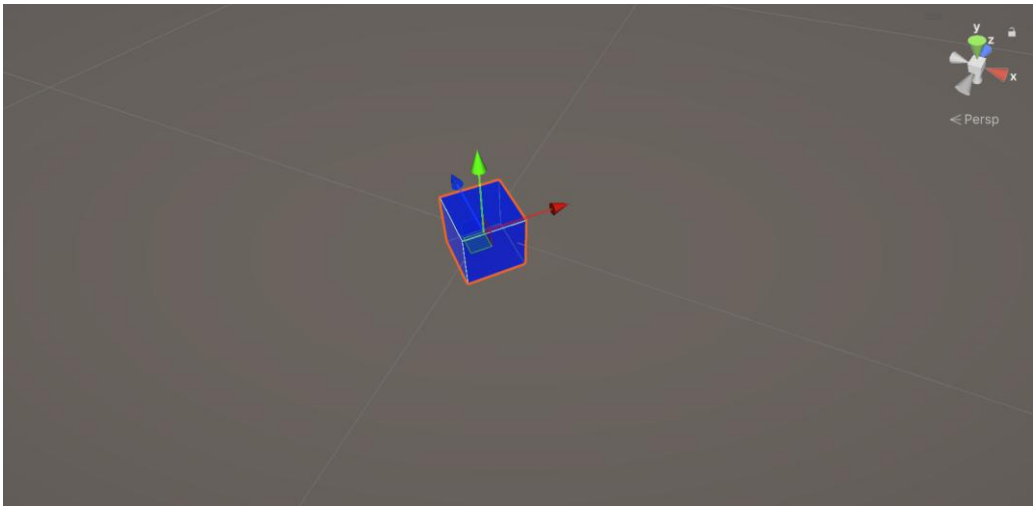

Figure 54 - GameObject's Transform in Local Space

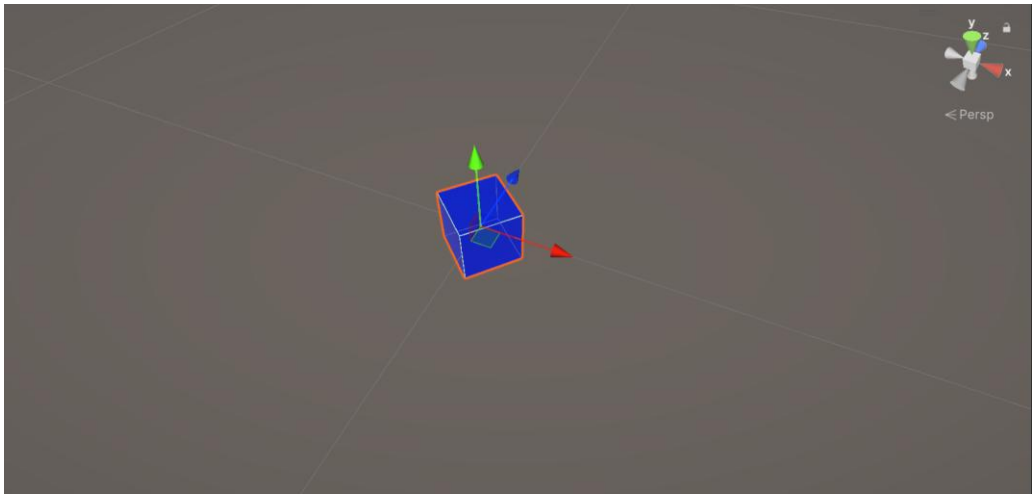

Figure 55 - GameObject's Transform in Global Space

Transformations of objects in 3D spaces contain intensive vector-based mathematics. To learn more about vector-based mathematics, I recommend taking MATH 149: Elementary Linear Algebra on campus.

One final note about local-space transformations: the three vectors Unity draws when moving the object are representative of your object's up (green), forward (blue), and right (red) directions, which is handy to know when programming your object's behaviour.

Visual Debugging

Visual debugging in Unity's scene view is primarily used when running your game. Unity uses what it calls a gizmo to represent objects in your scene. You may already be familiar with gizmos without realizing it. Unity draws a light and camera gizmos in your default scene, representing your directional light and main camera object, respectively.

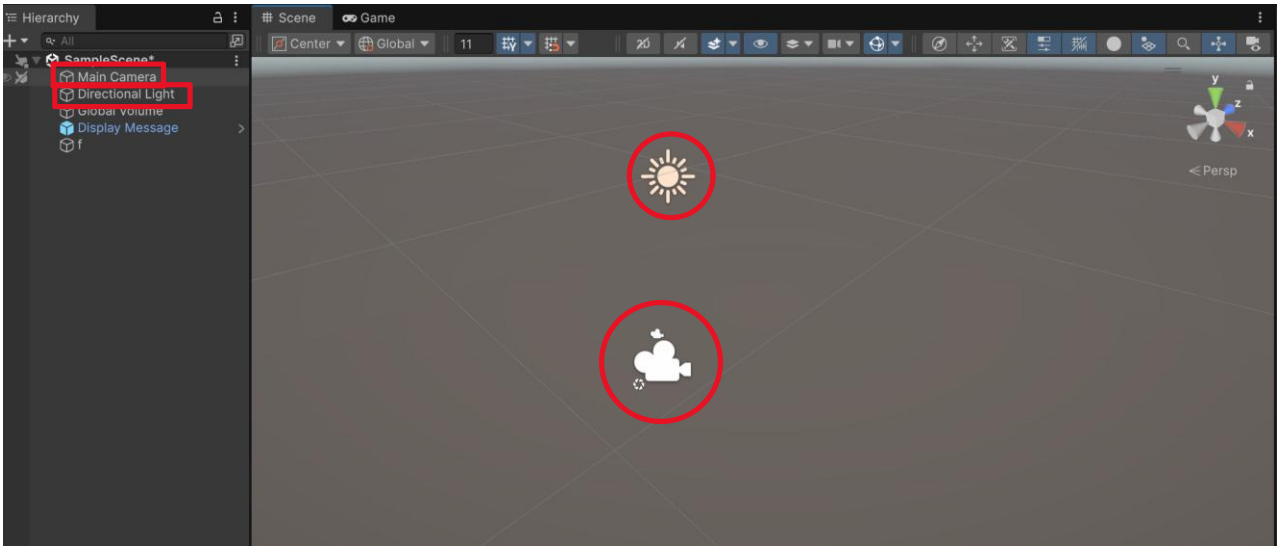

Figure 56 - Connecting Scene Gizmos to Their Game Objects

Unity has many types of gizmos, including:

- Cameras
- Lights
- Colliders
- Animations
- Navmeshes
- UI
- And much, much more

You can view the full list of gizmos by clicking the dropdown arrow next to the sphere icon in Unity's scene ribbon:

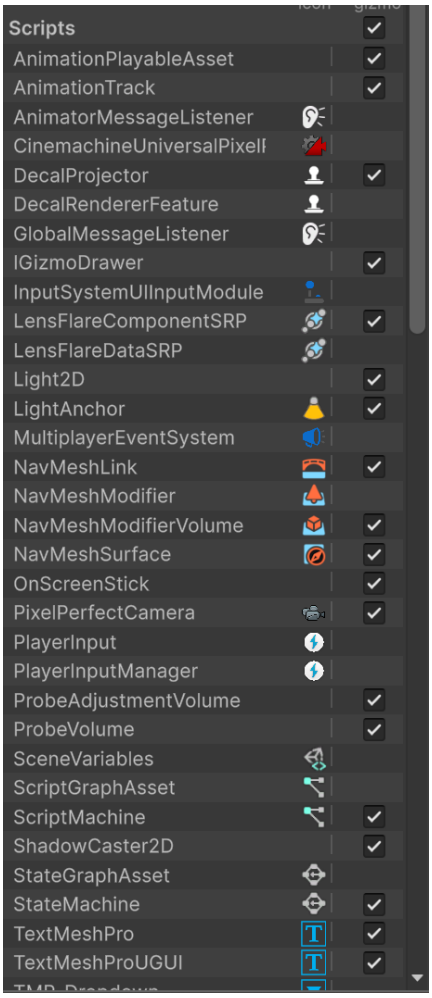

Figure 57 - Full List of Enabled Gizmos Settings

Creating Your Own Gizmos

While Unity supports built-in gizmos, it does not inherently support code debugging. However, Unity has a built-in event called OnDrawGizmos that lets you draw your own gizmos in code. OnDrawGizmos happens whenever your scene needs to be re-rendered, whether due to scene updates, scene camera movement, object selection, editor refreshes, and more. This makes OnDrawGizmos great for visually debugging complex systems, such as pathfinding, enemy detection range, hidden meshes, rays, collision detection, and more.

For example, say you have an enemy in your scene and want it to pathfind from its current position to a target position. You want to see the enemy's path in the world, but there are no supported gizmos that draw an enemy's path for you.

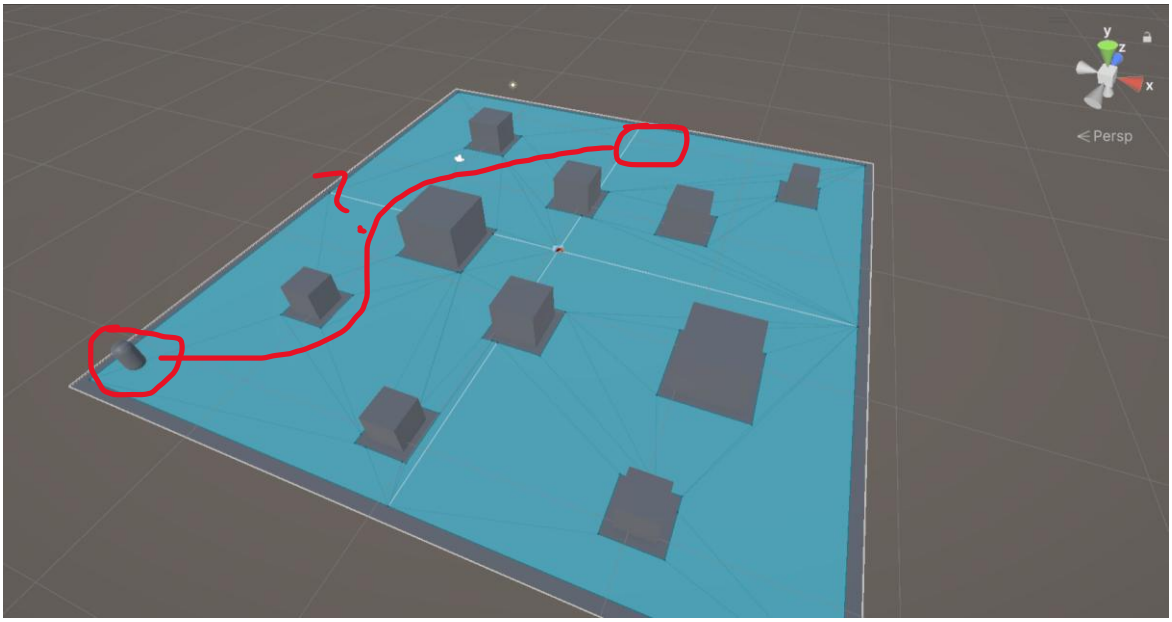

Figure 58 - Sample Layout of Object Pathfinding without Custom Gizmos

This is where you can use Unity's OnDrawGizmos event to draw the path for you. The object in the bottom-left corner has a custom pathfinding script attached to it. In that script, we can call the OnDrawGizmos Unity event to manually draw the object's path:

```
Unity Message | 0 references
private void OnDrawGizmos()
{
    //If no corners, then there's no path to draw
    if (currentPath.corners.Length == 0) return;

    //Set the gizmos color for how the path will appear in the scene
    Gizmos.color = Color.yellow;

    //Store the previous corner in the path to draw from that corner to the current corner
    Vector3 previousCorner = currentPath.corners[0];

    //Iterate through each point in the path, and draw a line from that to the previous point
    for (int i = 0; i < currentPath.corners.Length; i++)
    {
        //If first or last corner, then there's not enough corner pieces to draw a line
        if (i == 0 || i == currentPath.corners.Length - 1) continue;

        //Draw a line from the previous corner to the current corner
        Gizmos.DrawLine(previousCorner, currentPath.corners[i]);
    }
}
```

Figure 59 - Custom Gizmos Code To Draw The Object's Path

When we run the game now, we can see the yellow path our object will follow:

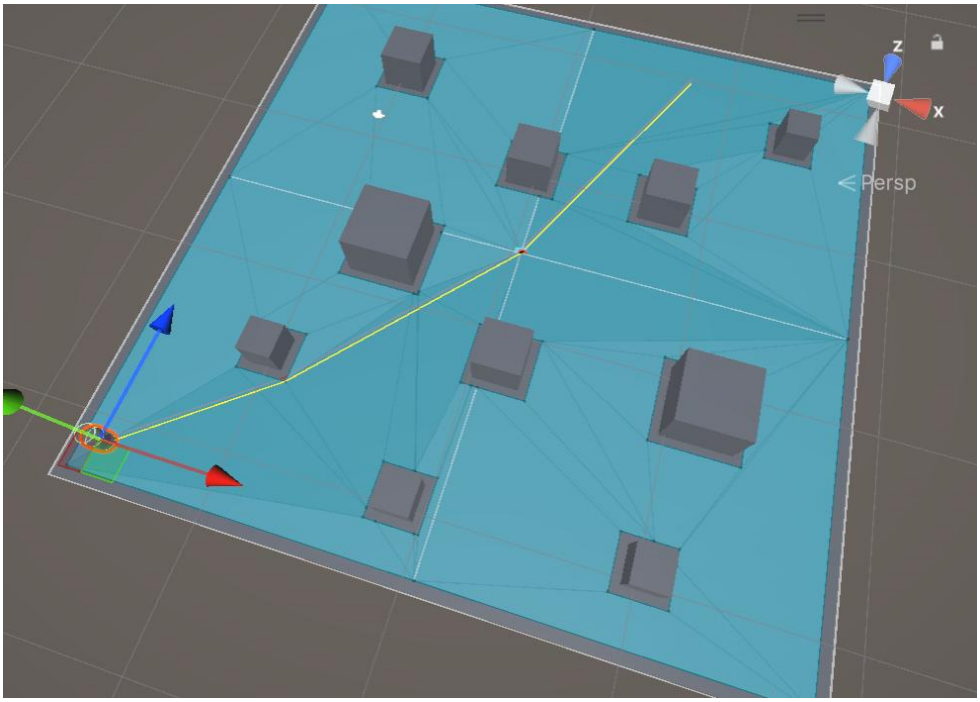

Figure 60 - Sample Layout of Object Pathfinding with Custom Gizmos

Scene Settings

Scene settings refer to the many configurable settings on the scene ribbon. There are many scene settings, and it's a common mistake to enable or disable a setting that causes your scene to stop operating as you wanted or expected. Let's break down each subsection of the ribbon:

The Overlay Menu

The overlay menu settings are located on the right-hand side of the ribbon. The overlay menu settings, from left to right, are:

- Toggle AI Navigation menu
- Toggle game object tools menu
- Toggle tool contents on the scene ribbon
- Toggle tool settings on the ribbon
- Toggle grid and snap settings on the ribbon
- Toggle draw mode settings on the ribbon
- Toggle view options on the ribbon
- Toggle search bar for game objects on ribbon
- Toggle orientation display in scene view
- Toggle camera overlay in scene view

Figure 61 - Location of Overlay Menu Settings on The Scene Ribbon

Many of the overlay menu settings control whether other scene settings are shown or hidden. If there is a setting you know is missing, look under the overlay menu section of the scene ribbon to toggle the setting's container on.

The View Options

The view options section of the scene ribbon refers to everything you see and how you see it in scene view. The settings from left to right are:

- Toggle 2D/3D view
- Toggle audio when running the game
- Toggle skybox, fox, and various other effects
    - Open the dropdown to select what exactly you want to/don't want to see in the scene
- Toggle visibility of hidden objects
- Select which layers to show in scene view
- Configure scene camera settings
- Toggle gizmos in your scene view
    - Open the dropdown to select the exact gizmos you do and don't want to see in scene view

Figure 62 - Location of View Options on The Scene Ribbon

Grid and Snap Settings

The grid and snap settings let you control the visibility and size of the scene's grid view. Grid settings are helpful when setting up a tile-based game. From left to right, the settings are:

- Set the length, width, and height for each grid tile uniformly
- Toggle grid visibility in scene view
    - Opening the dropdown lets you set the plane the grid will follow and the grid opacity
- Toggle object grid snapping
    - Opening the dropdown lets you better configure the snap settings on the x, y, and z axes. Also, it lets you set the rotation and scale increments when grid snapping.

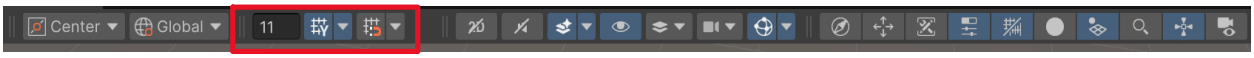

Figure 63 - Location of The Grid and Snap Settings on The Scene Ribbon

Tools Settings

The tools settings on the scene ribbon let you control how object transformations happen (see A Quick Note About Object Pivot & Orientation Settings). From left to right, these settings are:

- Toggle between center/pivot transformations
- Toggle between global/local transformations

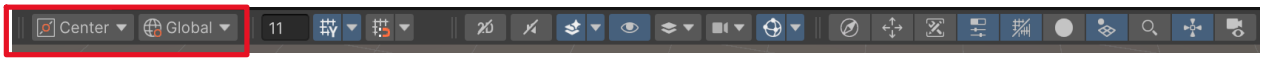

Figure 64 - Location of The Tools Settings on The Scene Ribbon

The Inspector

The inspector is Unity's way of showing an asset's properties in a single place. You can inspect almost every asset in Unity, so you will want to have quick access to the inspector window at all times. You will mostly use the inspector for the following assets:

- Game Objects/Prefabs
- Materials
- Scriptable Objects

Game Objects in The Inspector

Remember that everything in your scenes are game objects. Game objects behave differently due to Unity's entity-component system. Game objects in your scene can have many components attached to them, and you can view all of your object's components through the inspector. For example, take the following sample cube:

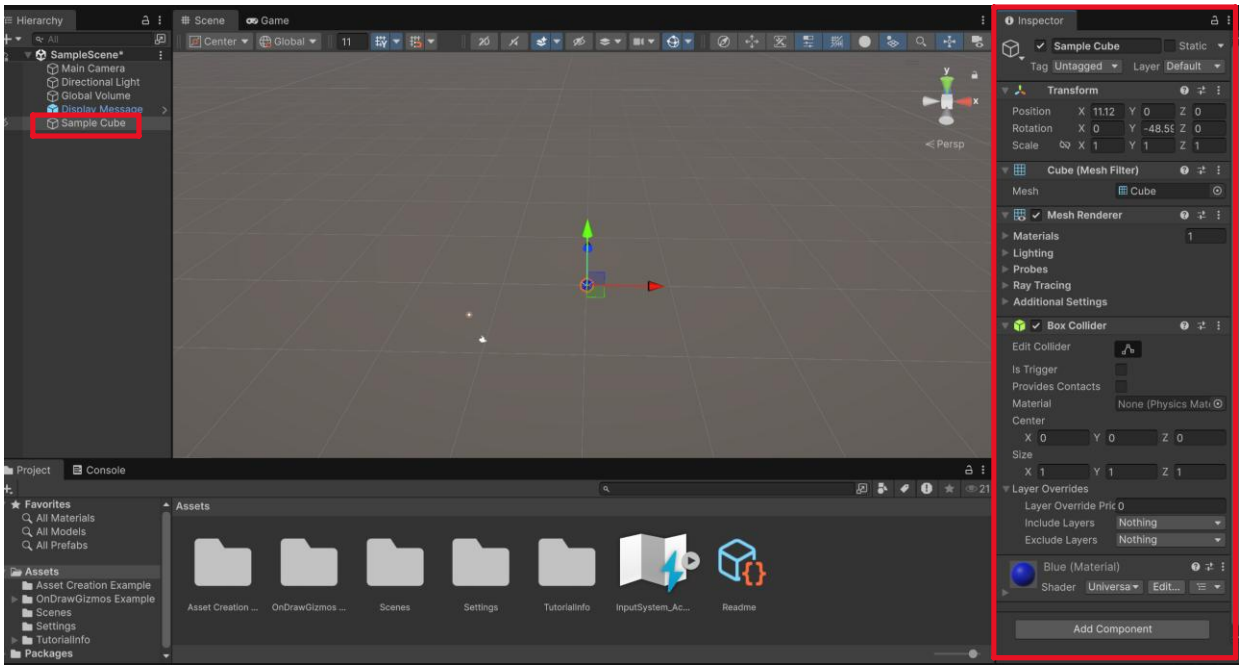

Figure 65 - Sample Inspector View of a Game Object

The sample cube includes a transform, a cube (mesh filter), a mesh renderer, and a box collider component. Additionally, it is using the blue material created earlier in this document.

Every game object has a transform component, and it will always be located at the top of the object's inspector. The transform component is the only required component for a game object. Every other component is optional to define the object's functionality and visuals.

You can add components to game objects through the inspector. You can do this by clicking the add component button at the bottom of the object's inspector.

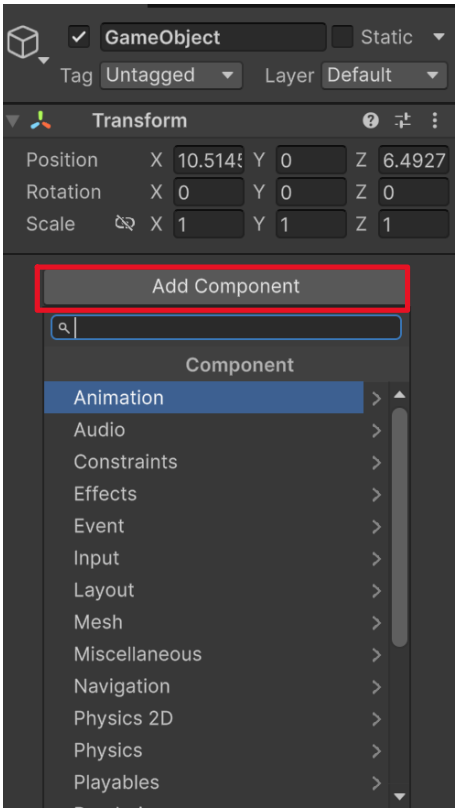

Figure 66 - Location to Add Components on a Game Object

Unity has hundreds of components pre-built for you. These components work with many of Unity's systems to fast-track your game's development process. Unity's pre-built components include:

- Rendering components
  - Mesh filter
  - Mesh renderer
  - Skinned mesh renderer
  - Sprite renderer
  - Tilemap renderer
- Lighting components
  - Light
  - Reflection probe
  - Light probe group
  - Lens flare
- Camera components
  - Camera
  - Cinemachine camera components
- Physics components

    - Rigidbody (2D & 3D)
    - Colliders (2D & 3D)
    - Constraints & joints (2D & 3D)
    - Character controller
- Audio components
    - Audio source
    - Audio listener
    - Camera listener (attached to main camera)
- Animation components
    - Animator
    - Animation controller

This is just a brief list of Unity's component systems. To learn more about Unity's entity-component system, check out their official documentation here.

Materials in The Inspector

Materials shape how you perceive 3D objects in your world, ranging from color to texture to transparency. Materials in Unity are broken up into smaller components, each of which defines how your material appears on game objects. The most important section to be concerned about with materials is the material's surface inputs, which are composed of a base map, metallic map, normal map, height map, occlusion map, and emission.

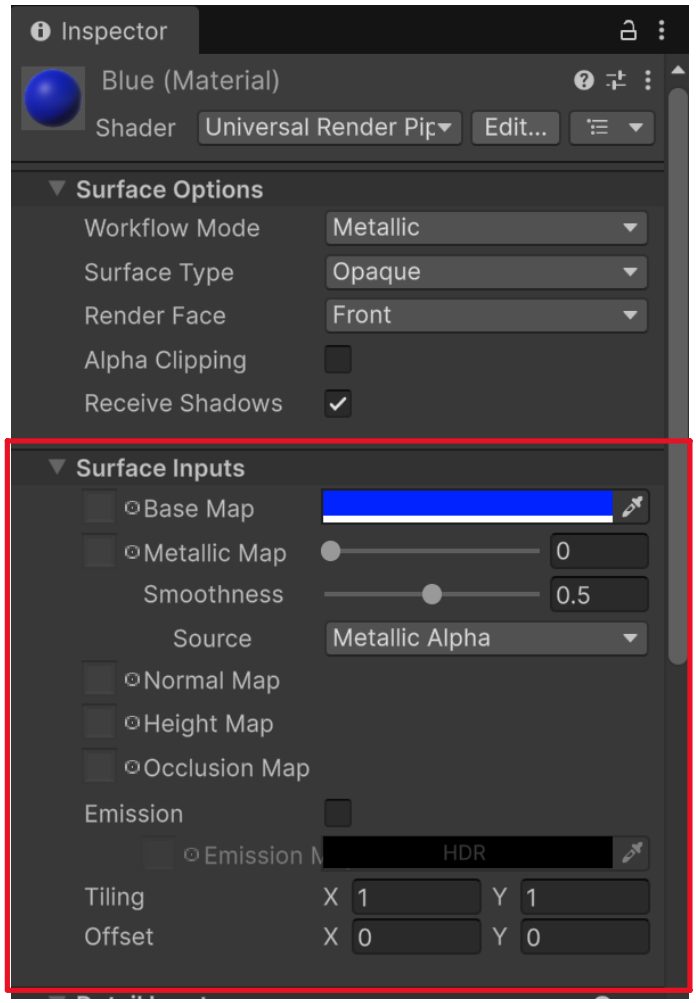

Figure 67 - Location of a Material's Surface Inputs in The Material's Inspector

Base Map

The base map is your material's color. The color can be singular, or, if you have an albedo texture file, it can be multicolored. Think of the base map as the general coloring of your object, devoid of depth or smoothness.

Figure 68 - Base Map Texture Example

Metallic Map

The metallic map determines whether your material will appear more like a metal or more rough. It controls how reflections behave, with a higher metallic value increasing reflections and a lower value reducing reflections. Other engines will refer to the metallic property of a material based on its roughness, like in Blender. The roughness is just the inverted value of the metallic value.

$$roughness = 1 - metallic\ value$$

$$metallic\ value = 1 - roughness$$

When importing a metallic texture into Unity, be sure that it is indeed a metallic texture and not a roughness texture. If it is a roughness texture, you need to invert the texture before applying it to the metallic map.

Figure 69 - Sample of Metallic (left) and Rough (right) Textures

Normal map

The normal map defines your material's depth. It is used to break up the flatness of the material and make it look like it has crevices in the object itself. It's important to know that the normal map does not change the actual shape of the object to which the material is applied. The normal map creates the illusion of depth through shading.

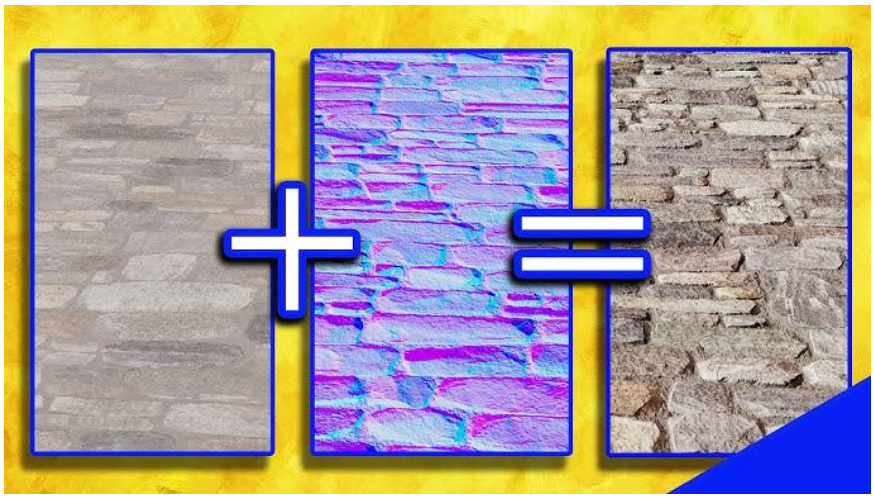

Figure 70 - Sample of a Base Map (Left) and a Normal Map (Middle) combined into a Material (Right)

Height Map

Unlike a normal map, a height map can produce the illusion of depth more accurately from any direction you look at an object, at the cost of performance. Additionally, you can use height maps to apply true surface deformations. However, this is more common when using the high-definition rendering pipeline (HDRP). Height maps store elevation data, whereas a normal map stores data on how light should reflect off the object's surface.

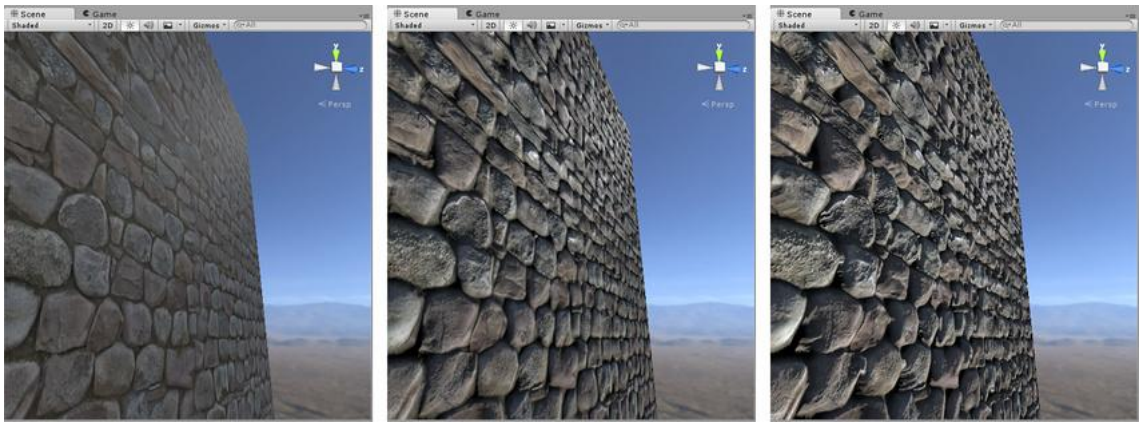

Figure 71 - Comparison of a Normal Map (Left) to a Height Map (Middle), and a Height Map at a Different Angle (Right)

Occlusion Map

The occlusion map darkens areas on the material where light should naturally be blocked. For example, a T-shirt has many creases in it, and you can naturally see these as darker areas on the shirt. The occlusion map stores the area on the material that should be occluded.

Figure 72 - Sample Creases (Darker Areas) in a T-Shirt

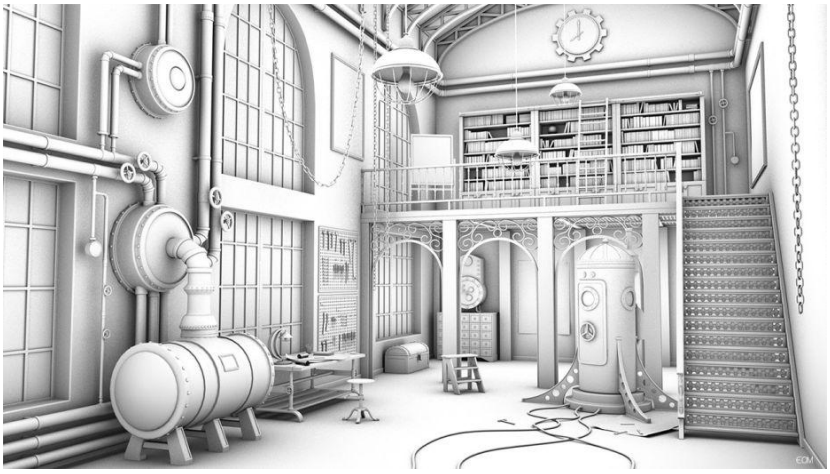

Figure 73 - Detailed Occlusion of a Custom Room

Emission

Emission on a material defines how the object glows. Unlike light components, emission only affects how bright the material appears; it cannot illuminate other nearby objects.

Objects with emission appear to glow, like a bright light bulb. In Unity, you can control how much a material glows by the emission's intensity property.

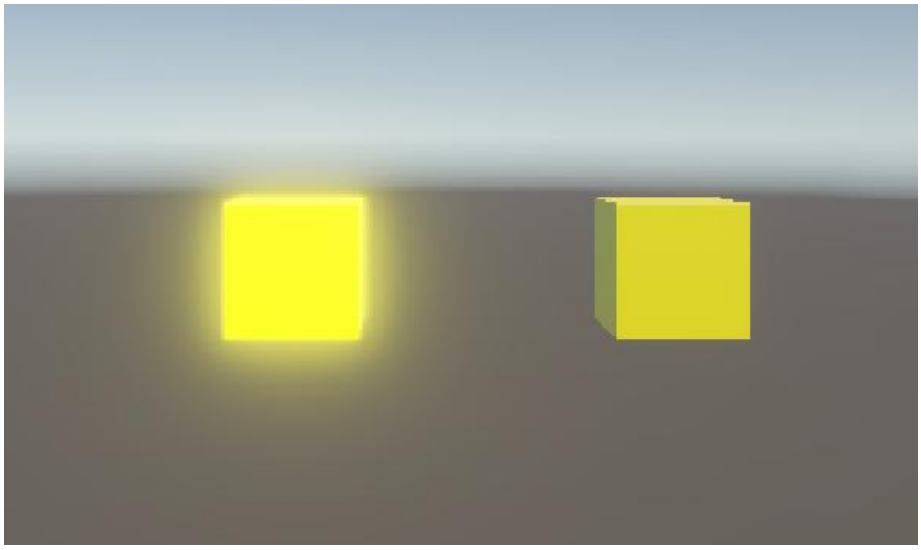

Figure 74 - Sample Materials with High Emission (Left) and Low Emission (Right)

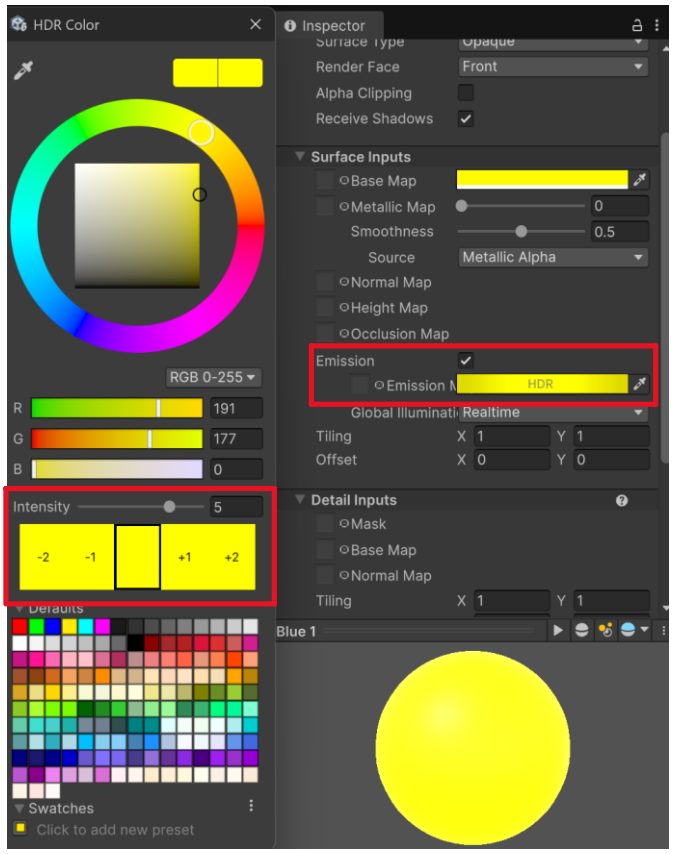

Figure 75 - Location of Emission Intensity in The Material's Inspector

Scriptable Objects in The Inspector

Scriptable objects are custom data assets. They only exist in Unity (although other game engines have the equivalent of a scriptable object), and they primarily store data. Scriptable objects are C# classes that *inherit* from the ScriptableObject class. Scriptable objects can store data about anything you want. For example, say you have an enemy system in your game. An enemy would have a max health, attack damage, attack speed, and attack radius attribute (you can add more or less if necessary). It's easy to store this information in a scriptable object asset and then retrieve the data from the asset when necessary.

Creating a Scriptable Object

To create a scriptable object:

1. Create a new C#/MonoBehaviour script in your project asset window, and name it "EnemySO". Open your script.
    a. Note: scriptable object scripts are typically marked with the suffix "SO". This is to make it easier to distinguish scriptable object scripts from gameplay and other scripts.

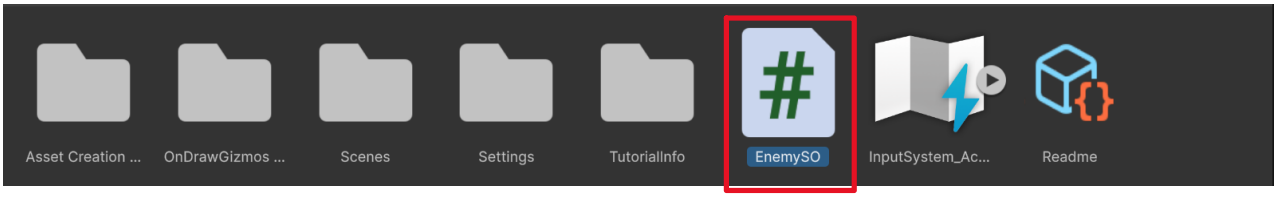

Figure 76 - Viewing Custom Scriptable Object Script in The Project Window

2. Apply the following changes:
    a. Change MonoBehaviour to ScriptableObject
    b. At the top of the class declaration, create an asset menu that defines the file name and menu name (see below).
    c. Add a public integer variable for max health and attack damage
    d. Add a public float variable for attack speed and attack radius
    e. Save your script (CTRL + S).

```
using UnityEngine;

//The file name defines the deafult name of the asset when created in the inspector
//The menu name defines where you can create this scriptable object in the create menu in the inspector
[CreateAssetMenu(fileName = "New EnemySO", menuName = "ScriptableObject/EnemySO")]
Unity Script | 0 references
public class EnemySO : ScriptableObject
{
    public int maxHealth;
    public int attackDamage;
    public float attackSpeed;
    public float attackRadius;
}
```

Figure 77 - Starter Setup of a New Scriptable Object Script

3. In Unity's project asset window, Right-click->Create->ScriptableObject->EnemySO. Name your scriptable object.

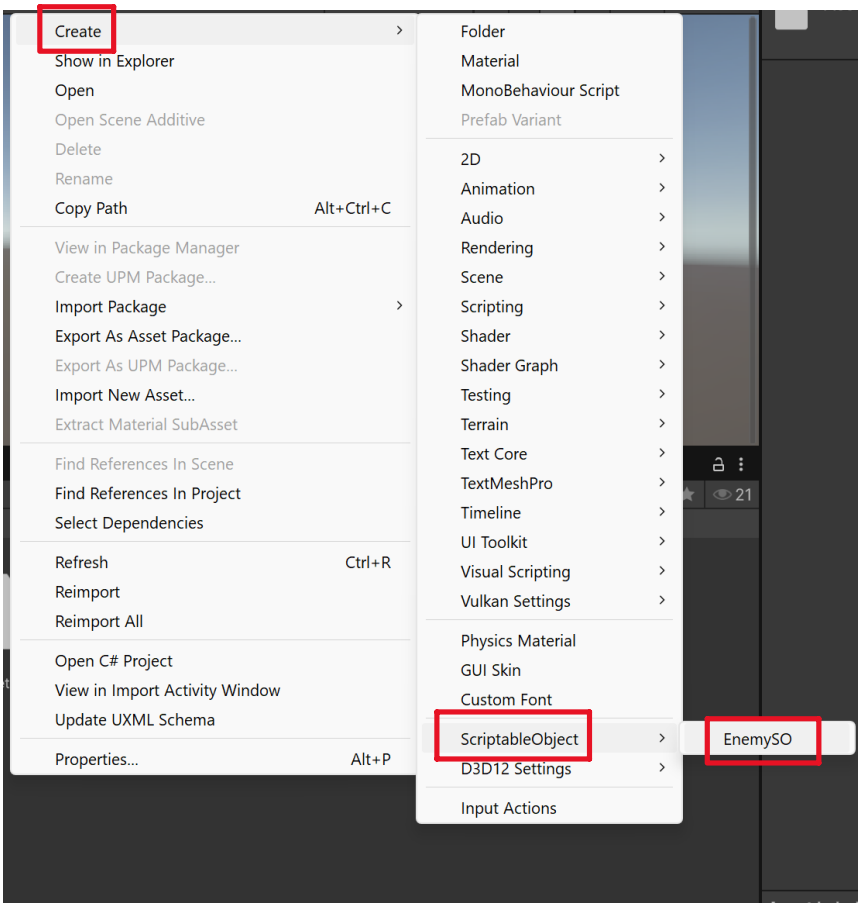

Figure 78 - Location to Create a New Scriptable Object Asset

4. Look at the inspector. You can see all the attributes defined in your script. Set them to values of your choosing.

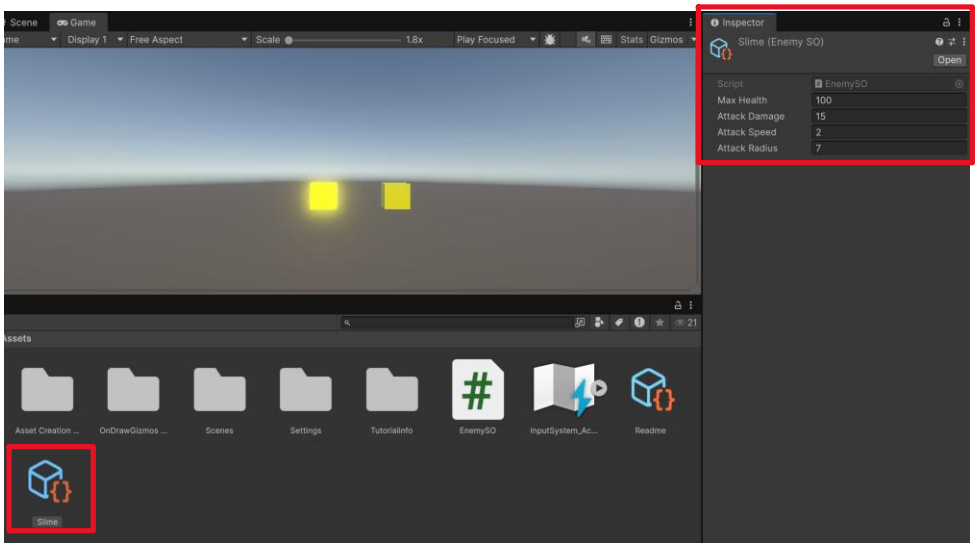

Figure 79 - Viewing The Custom Scriptable Object's Properties in The Inspector

After creating your scriptable object asset, you can reference the asset in your gameplay scripts and use its values for gameplay logic. You can create as many scriptable object assets as you want. Just be sure to reference the assets in your scripts.

The Menu Bar

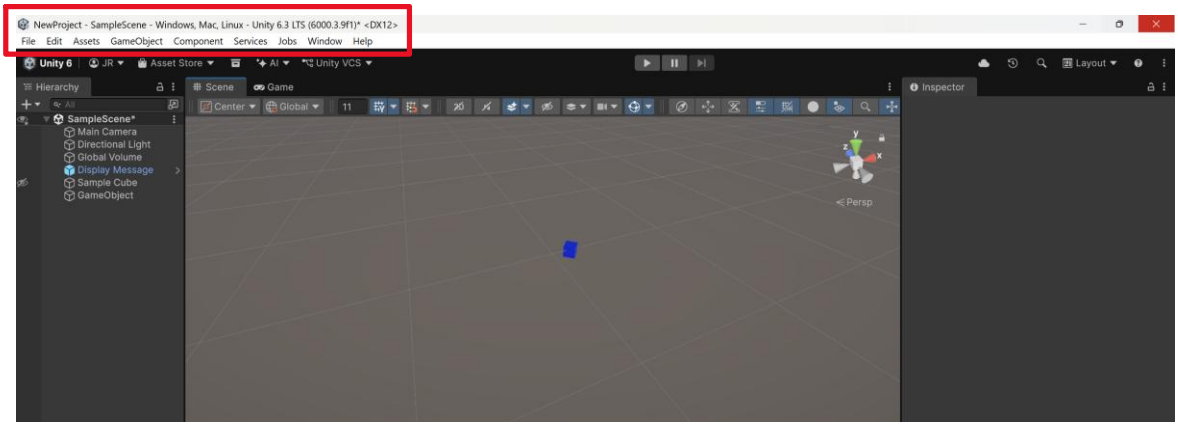

Figure 80 - Location of The Menu Bar in The Unity Editor

The menu bar contains many project-wide actions for your game. It exposes many actions related to your scene, build, assets, and editor tools. Below, you may find some of the most important options in the menu bar:

Build Profiles/Build Settings

The build profiles window (legacy editor versions call it build settings) is where you can configure how your project will be built. You can access it via File->Build Profiles.

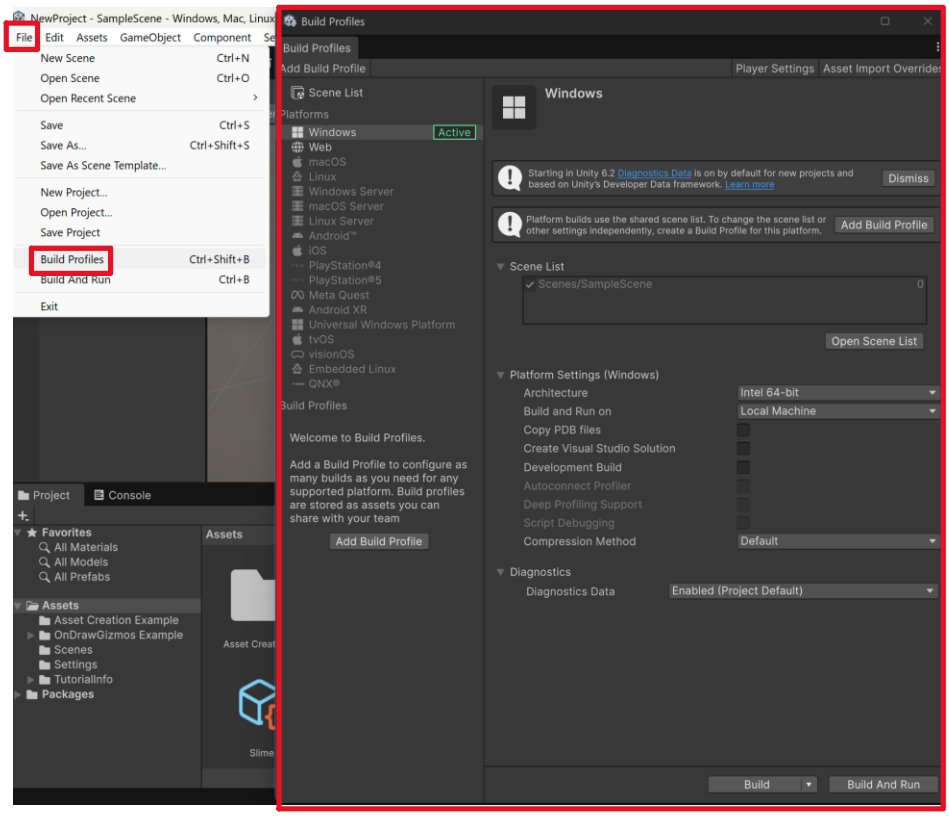

Figure 81 - Location of The Build Profiles/Settings in The Menu Bar

In the build profiles, you can choose the platform (the left panel) you wish to build your project on. Additionally, it has the scene list, which stores a list of all the scenes your game will be built with. Any scene that is not included in the build profiles' scene list will not be accessible in the built version of your game. Additionally, you will not be able to switch active scenes via code unless they are added to the scene list.

You can add a scene to the scene list by clicking the open scene list button and dragging and dropping your scene asset into the list. Do know that the scene at the top of the list will be the scene Unity opens the game to.

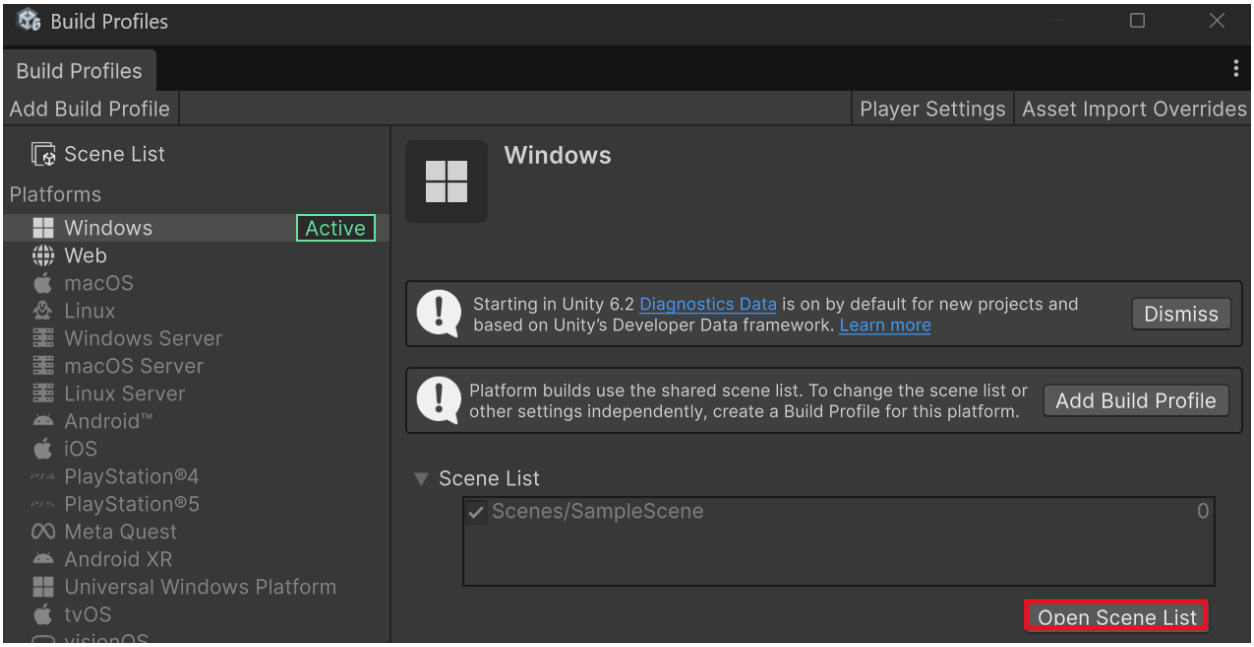

Figure 82 - Location to Add a Scene to The Scene List

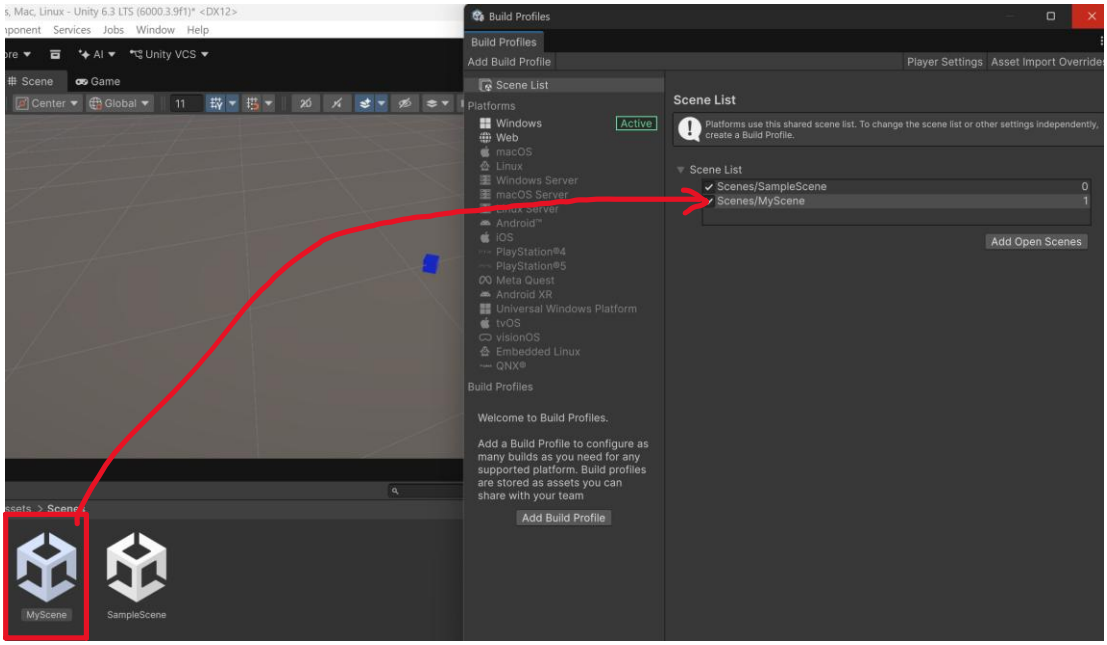

Figure 83 - Dragging and Dropping a Scene Asset Into The Scene List

Project Settings

The project settings window is where you can configure many settings at the project level. You can access the project settings via Edit->Project Settings.

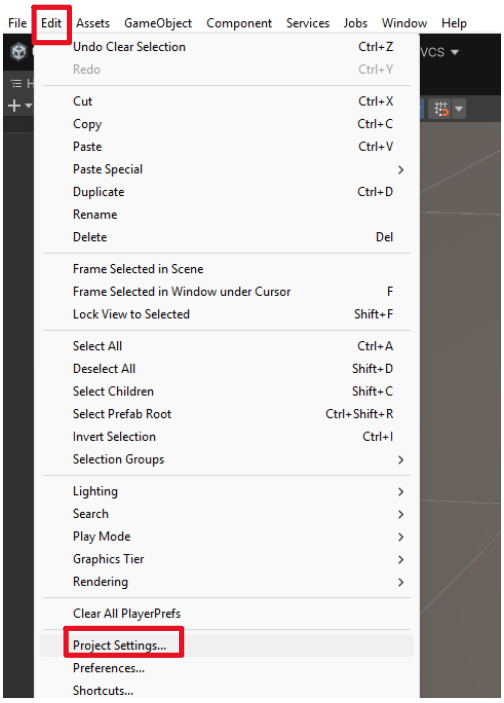

Figure 84 - Location of The Project Settings in The Menu Bar

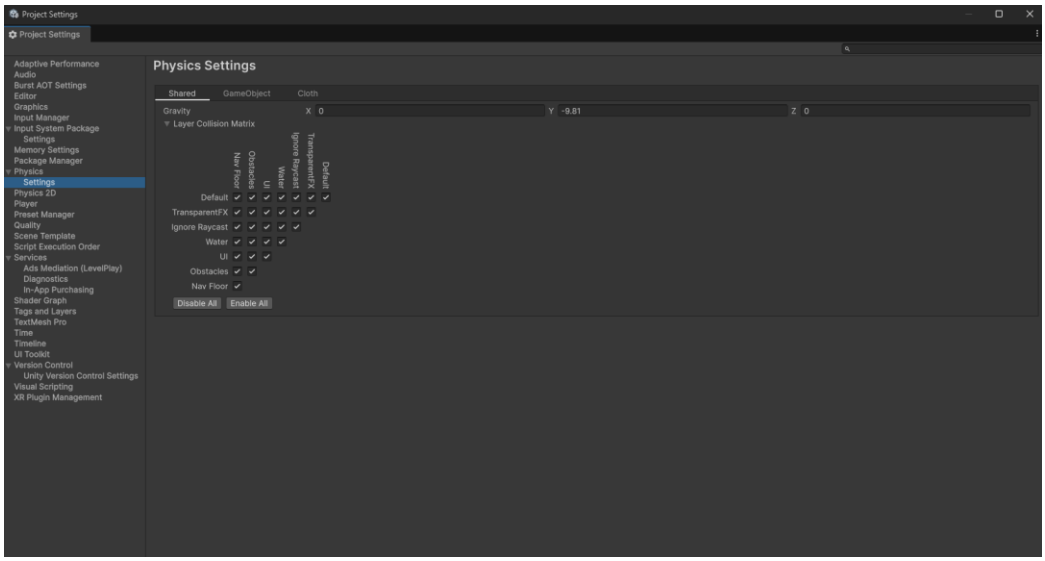

Figure 85 - Sample View of The Project Settings Window

There are a lot of project settings. However, you will find yourself coming back to the project settings for some settings more than others. This includes:

- Physics settings
- Script execution order
- Tags and layers

Physics settings

The physics settings control how objects in your scene will interact with one another. Additionally, it is where you can control the gravity of your project.

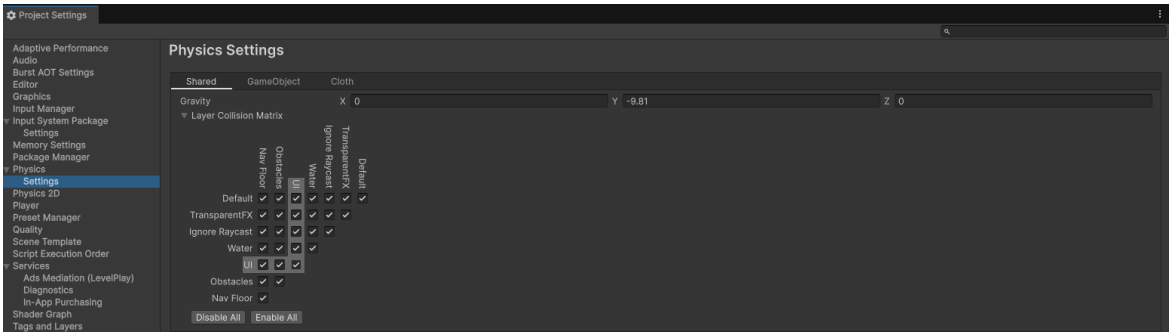

Figure 86 - Sample View of The Physics Settings in The Project Settings Window

Gravity in the project settings will apply a force to all rigidbody objects in the scene in the direction specified, and at the magnitude specified. By default, the gravity is set to -9.81 on the y-axis, and 0 on the x and z axes, simulating the effects of gravity we experience every day.

The physics settings expose the layer collision matrix, or a matrix defining every type of collision between all layers. By default, everything is checked to collide with each other. However, you can disable collision layers to prevent the two layers from colliding with one another. For example, the you can disable collision between two objects on the default layer by unchecking the checkbox corresponding to the default-default layer collision.

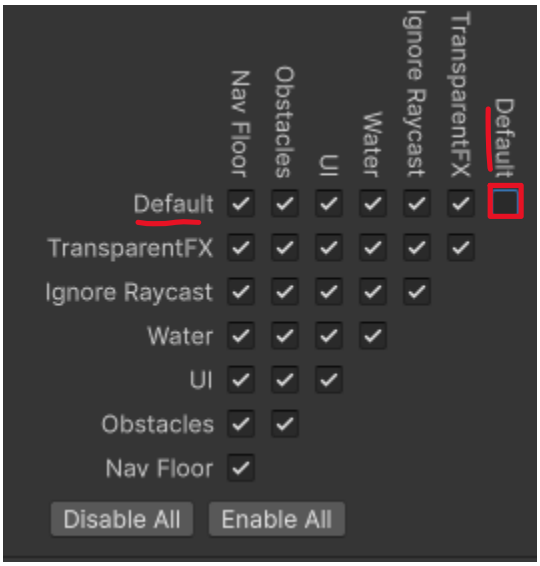

Figure 87 - Disabling Collision Between Two Layers in The Layer Collision Matrix

Now, when you run your game, objects on the default layer will not physically interact with one another. This is great for controlling collision behavior between objects. However, do know that disabling collision between two layers means the objects will not call OnCollisionXXX or OnTriggerXXX Unity events in code, so disable collision only if you're certain the two layers should never interact with each other.

Script Execution Order

The script execution order determines the order in which Unity will run your scripts. By default, all scripts you create are run during Default Time. This means that, by default, you have no control over the order in which your scripts run. This is a common problem when you have a script dependency, and one script must run before another.

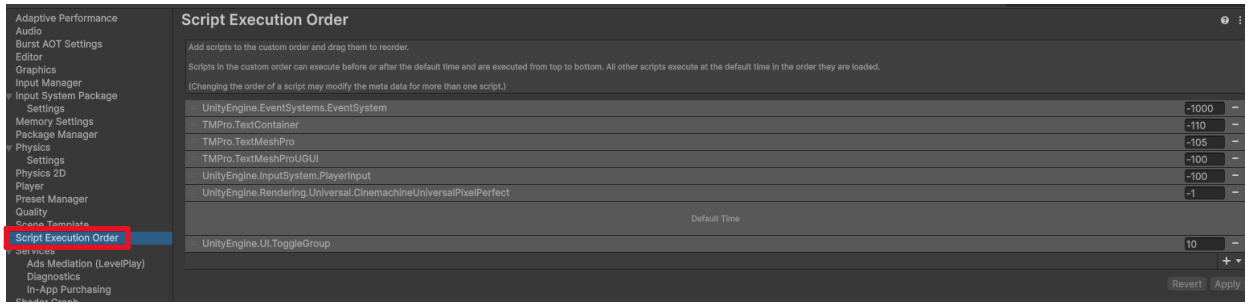

Figure 88 - Location of The Script Execution Order in The Project Settings

Take the following scripts attached to a game object in a scene: Foo and Bar. Foo and Bar will display their script name to the console when the game runs.

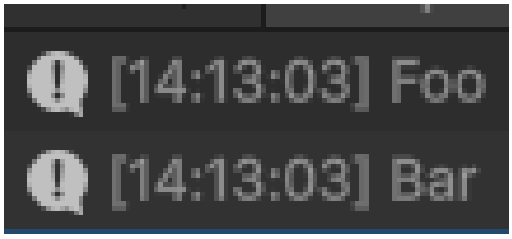

Figure 89 - Sample Output of Foo and Bar (Random Script Execution Order)

By default, Foo ran before Bar. Say Bar needed to run before Foo. This is where you can use script execution order to change the order. Click the + icon in the bottom-right of the script execution order, and add your Foo and Bar scripts. Rearrange them such that Bar comes before Foo.

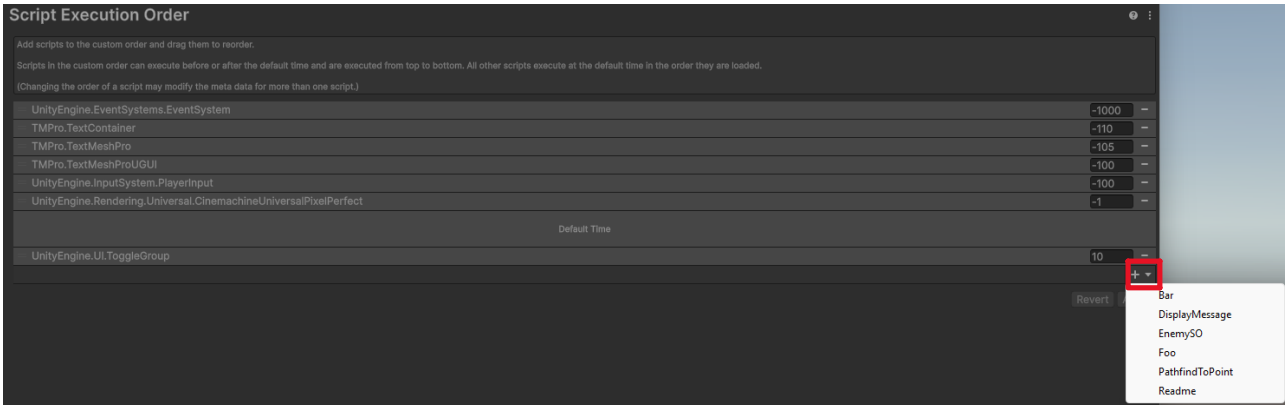

Figure 90 - Location to Manually Change a Script's Execution Order

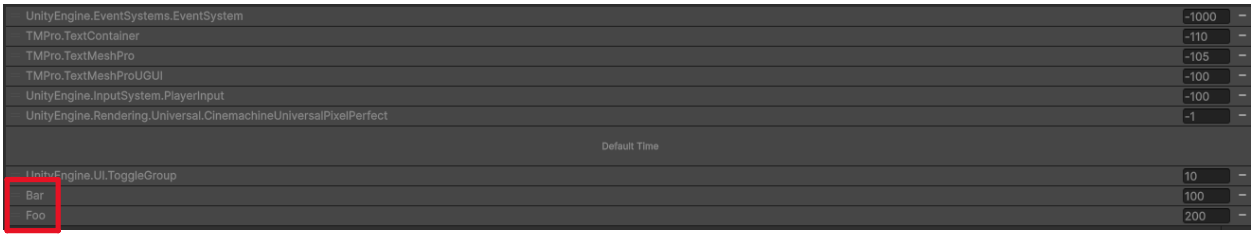

Figure 91 - Setting Up Bar.cs to Run Before Foo.cs

Now, when running your game, Bar will always run before Foo.

Figure 92 - Sample Output of Foo and Bar (Controlled Script Execution Order)

Tags & Layers

The tags and layers settings let you create or delete tags and layers in your game. You are limited to 32 layers per project, but you may make as many tags as you want. Additionally, you can create sorting layers for your project.

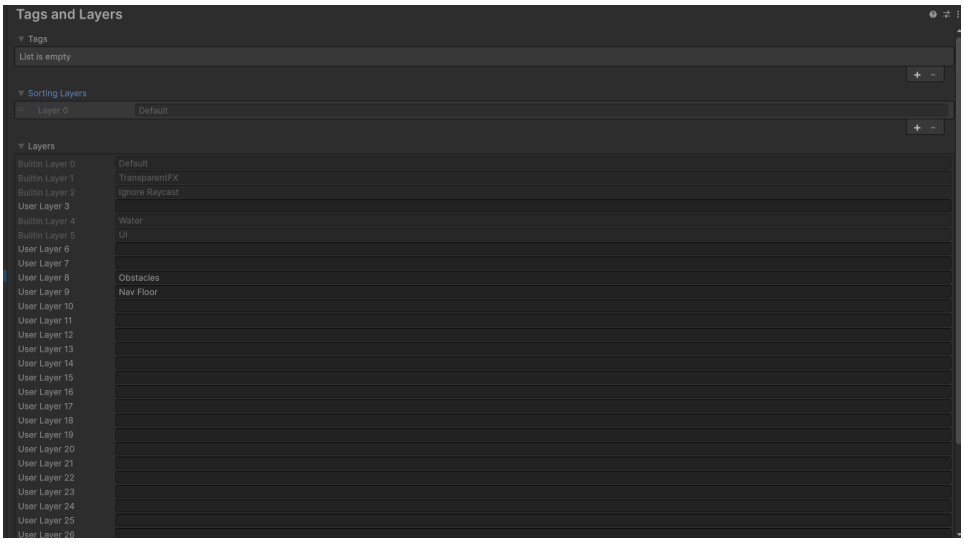

Figure 93 Sample View of The Tags and Layers in Project Settings

Tags and layers appear to have the same functional purpose, but they have some key differences. Tags are used to provide a form of object identification (i.e., player, enemy, projectile) and are used for gameplay logic. Layers, on the other hand, are used to control physics and rendering systems.

Package Manager

The package manager is the central hub for installing and uninstalling Unity packages in your project. You can access the package manager via Windows->Package Management->Package Manager.

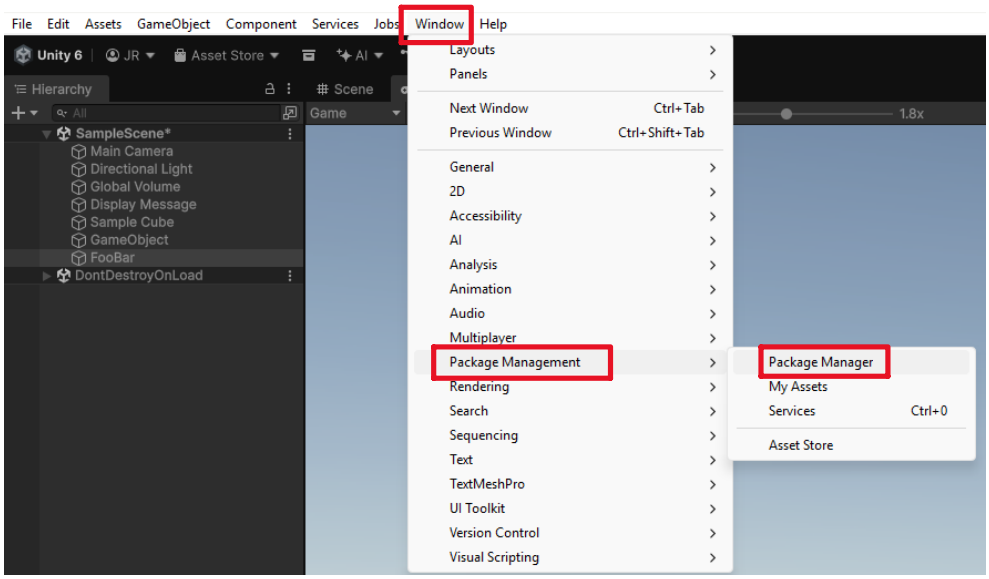

Figure 94 - Location of The Package Manager in The Menu Bar

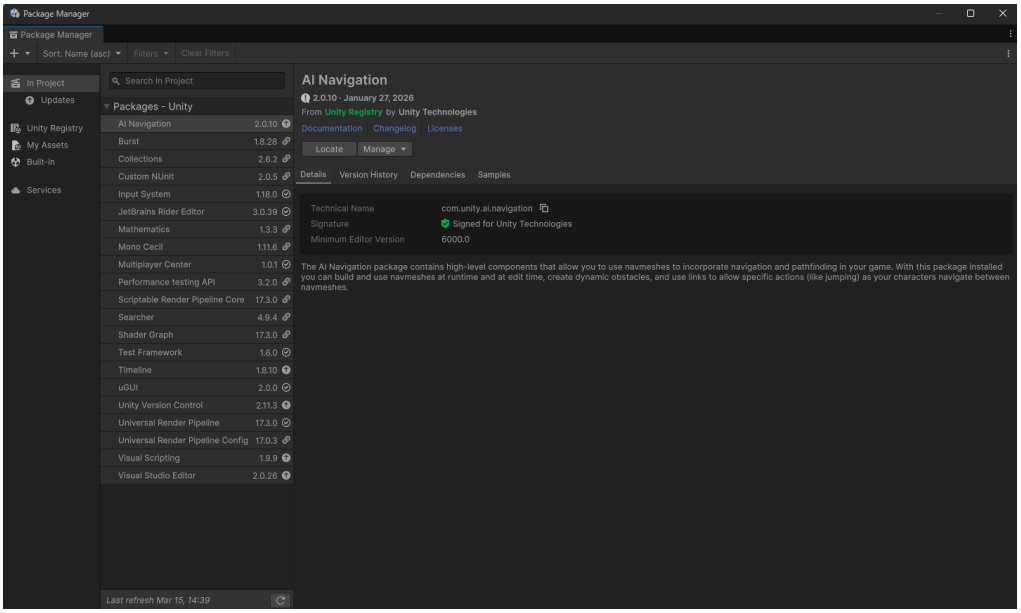

Figure 95 - Sample View of The Package Manager

The package manager shows you all packages installed on your project by default. However, you can view all of Unity's packages through the Unity registry section. Common packages you will install from Unity's registry are:

- TextMeshPro
- Input System (now installed by default)
- Cinemachine
- Netcode for GameObjects (for multiplayer projects)

Unity will automatically install the latest version of a package when you go to install one. If, however, you need to install an older package, you can do so by installing it by name:

1. Click the + button at the top-left corner of the package manager. Click "Install package by name..."

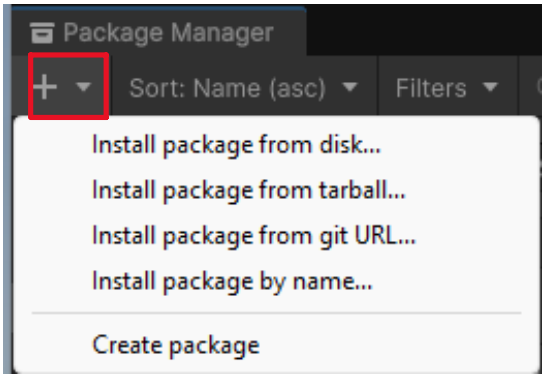

Figure 96 - Location to Install Packages by Their Name

2. Enter the name of the package (i.e., com.unity.2d.animation for the 2D animation package)
    a. Note: You can find the name of a package by finding it in the Unity registry and looking for the technical name field.

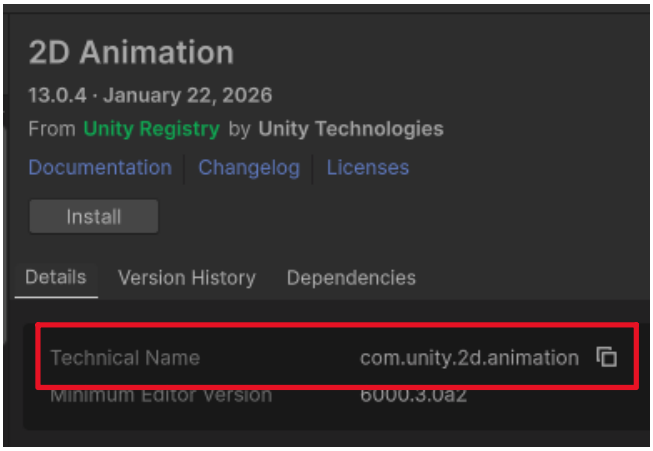

Figure 97 - Finding The Technical Name of Packages

3. Enter the package version you wish to install (i.e., 13.0.4).
4. Click the Install button

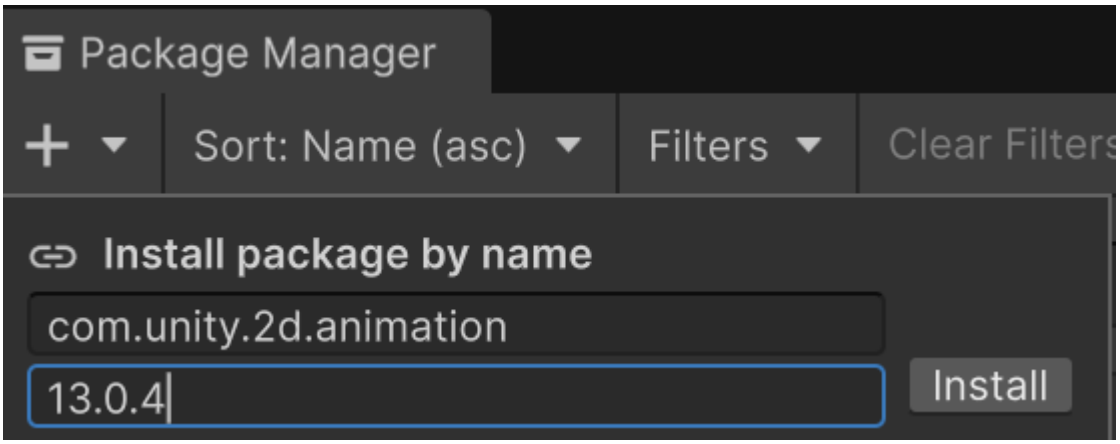

Figure 98 - Installing Packages by Their Name and Version

Other Useful Windows

Unity hides many relevant windows in the menu bar. Simply put, there are too many windows to have docked on your screen at once. You can still access all of Unity's windows, but you will need to find them in the window section of the menu bar.

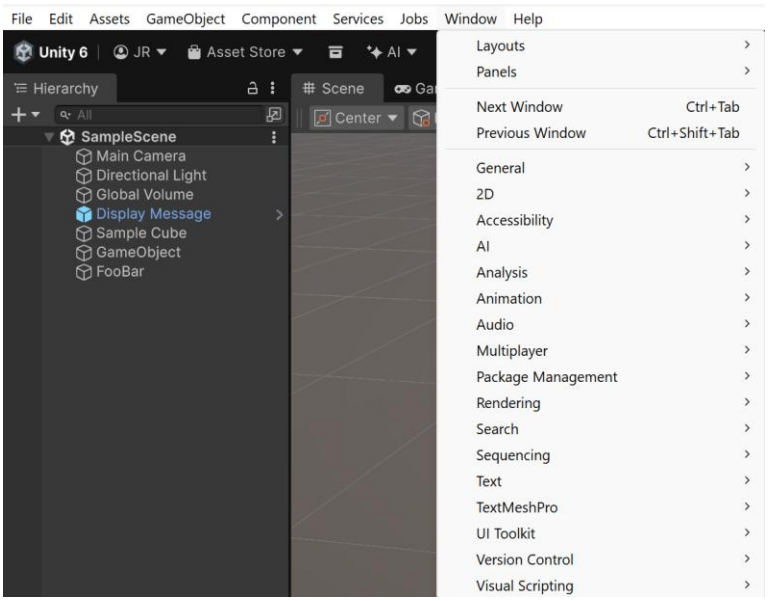

Figure 99 - Finding All Unity Windows in The Menu Bar

Below are some additional windows that are helpful:

- Animation (Window->Animation->Animation): Opens the window where you can animate game objects
- Animator (Window->Animation->Animator): Opens the window to control an object's animation flows and define animation states
- Multiplayer Center (Window->Multiplayer->Multiplayer Center): Opens the window that lets you quickly configure your project for multiplayer development
- Profiler (Window->Analysis->Profiler): Gives you real-time game performance to help identify bottlenecks in optimization

Docking Windows to Your Editor

Unity allows you to dock windows to your editor. You can drag and drop any window into any of the views in your editor, and Unity will add the window to that view as a tab. You are likely already familiar with docked windows without realizing it. For example, the scene and game window are both docked in the middle of the screen, and the project and console window are both docked at the bottom of the screen. You can do this for any window.

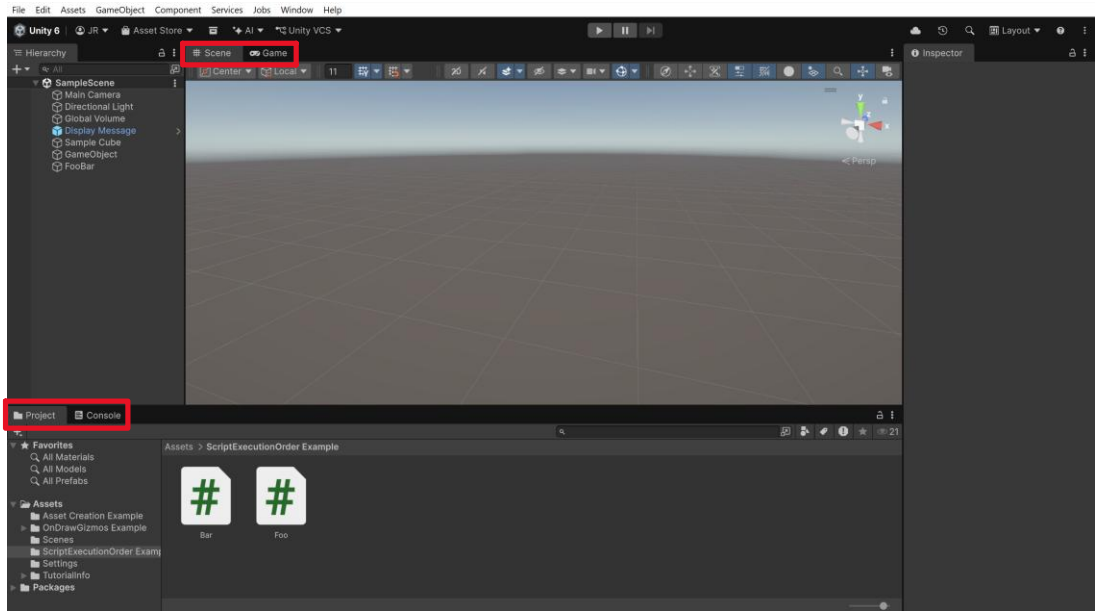

Figure 100 - Viewing Unity's Pre-Docked Windows in The Default Layout

Some common windows you will want to dock to your editor include:

- Build Profiles
- Package Manager
- Project Settings
- Animation & Animator

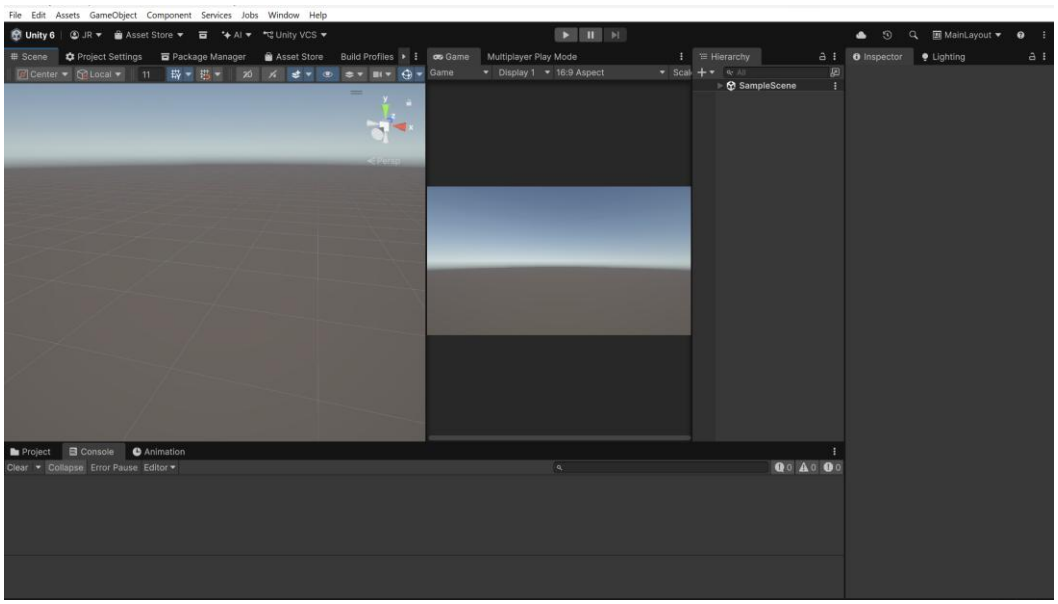

Figure 101 - Example Custom Layout Setup

Saving/Swapping Editor Layouts

Once you have a new editor layout, you will want to save it. If you forget to do so, you will lose your layout the next time you swap to a new one. To save your layout, find the editor layout dropdown at the top-right corner and click the “Save Layout…” button. Name your layout, and Unity will save your layout for you.

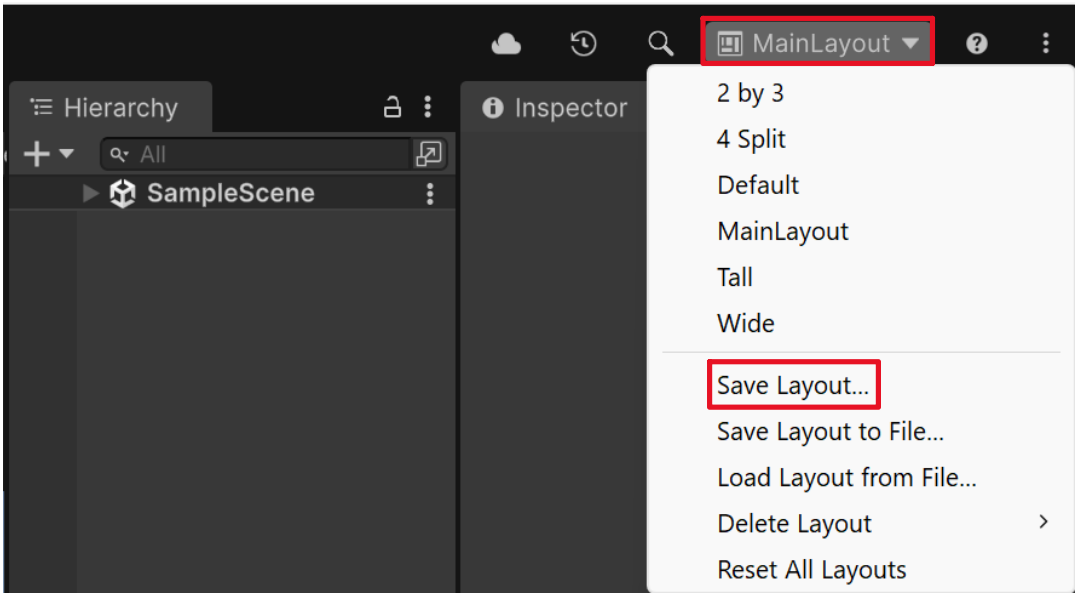

Figure 102 - Saving Custom Loadout to Unity

The area at the top of the editor layout dropdown is all of your layouts, some that you made and others that Unity made. You can use this to cleanly swap between different layouts. For example, you may have a layout for customizing animations, designing your game scenes, debugging systems, and configuring project settings and packages. Or, you can have one main layout that holds everything. The choice is yours.

References

https://docs.unity3d.com/Packages/com.unity.entities@6.5/manual/concepts-intro.html